

Un Peuple – Un But – Une Foi

**UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR**



**ÉCOLE SUPÉRIEURE POLYTECHNIQUE**

Département Génie Informatique

**MÉMOIRE DE FIN DE CYCLE**

Pour l'obtention du :

**MASTER EN GÉNIE INFORMATIQUE**

OPTION : Sécurité des Systèmes d'Information

**Promotion : 2025–2026**

**SUJET :**

# AFFIC : conception et déploiement d'une plateforme web sécurisée de gestion de portefeuille client

| | |
|---|---|
| **Lieu de stage :** | DataBeez, Dakar |
| **Période de stage :** | Mai 2026 – Octobre 2026 |



| | |
|---|---|
| **Présenté et soutenu par** | Arphan Kaback Junior BODIAN |
| **Encadrant** | Dr Doudou FALL, Maître-Assistant (CAMES) |
| **Maître de stage** | Mme Maguette TOURÉ |
| **Directrice de mémoire** | Dr Khadidiatou Wane KEITA (Maître de conférences du CAMES) |



**Année universitaire : 2025–2026**

# Dédicace

*Je dédie ce travail :*

*À mon père,* ***Tahirou Karamba Bodian****,*
*et à ma mère,* ***Khady Coly****,*
*pour leur amour inconditionnel, leurs sacrifices*
*et leurs prières qui m'ont accompagné tout au long de mon parcours ;*

*À mes frères et sœurs,* ***Boubou Dramane Bodian****,* ***Ousmane Samade Bodian****,*
***Malick Bodian****,* ***Wilfried****,* ***Aïssatou Coly*** *(BBJ),* ***Fatoumata Bodian****,*
***Marie Madeleine****,* ***Rose Marie*** *et* ***Winnie Amalia****,*
*pour leur soutien constant et leurs encouragements ;*

*À mes confrères de la team* ***Comlaab*** *: Meka, Ameth Sall, Idrissa Ba,*
*Elhadj Cisse, Lord (Lamine Faye), Amadou Sall Gueye*
*et tous les autres ;*

*À mes breudeu : Khady Sow, Aïssatou Lam, Faby Youm et Mame Sokhna ;*

*À* ***Meme Kante*** *;*

*À mes parrains, et à ma marraine* ***Mbissine*** *;*

*À mes filleuls : Elimane Ka, Rassoul Name, Raphaël, Pape Soce*
*et Ismaël Doukoure ;*

*À mes seuts : Ousmane Dieng, Thiombane, Seynabou Ndiaye,*
*Ndeye Maguette Ka, Oumar Ka et Jean François Martin Diouf ;*

*À mes promotionnaires : Yacine Diagne, Simon Kamate,*
*Alassane Latyr Sarr, Massina Bassene, et à toute ma promotion ;*

*À toutes mes mames, à tous mes filleuls et à tous mes seuts ;*

*À toute ma famille et à mes amis,*
*qui ont cru en moi et m'ont soutenu dans les moments difficiles.*

# Remerciements

Au terme de ce travail, je tiens à exprimer ma profonde gratitude à toutes les personnes qui, de près ou de loin, ont contribué à sa réalisation.

Mes remerciements s'adressent en premier lieu à ma directrice de mémoire, Dr Khadidiatou Wane KEITA, pour la qualité de son encadrement, sa disponibilité, ses conseils avisés et sa rigueur scientifique, qui ont largement contribué à l'aboutissement de ce mémoire.

J'adresse également mes sincères remerciements à mon encadrant, Dr Doudou FALL, pour son suivi attentif, sa disponibilité et ses remarques constructives tout au long de ce travail.

Je remercie très sincèrement ma maître de stage, Mme Maguette TOURÉ, pour son accueil, sa confiance et son accompagnement tout au long du stage, ainsi que pour ses orientations qui ont guidé la conduite de ce projet.

Mes remerciements vont aussi à M. Sidaty KANE pour sa disponibilité, ses conseils et son soutien tout au long de ce travail.

Je remercie l'ensemble du corps professoral et administratif de l'École Supérieure Polytechnique de Dakar, en particulier les enseignants du Master en Génie Informatique, option Sécurité des Systèmes d'Information, pour la qualité de la formation dispensée durant ces années d'études.

Je remercie également la direction et les équipes de DataBeez, qui m'ont accueilli et fait confiance pour la conduite de ce projet, ainsi que les collaborateurs du cabinet AFFIC pour leur disponibilité lors des phases de recueil des besoins et de validation de la solution.

J'adresse une pensée toute particulière à mon parrain, Wilfried DZADOU, et à ma marraine, Mbissine, pour leur bienveillance, leurs conseils et leur soutien constant, ainsi qu'à mes promotionnaires Abdoulaye Barro, Ameth Sall, Mohamed Ka et Idrissa Ba pour leur aide et leur camaraderie tout au long de ce parcours.

Mes remerciements vont enfin aux membres du jury, qui ont accepté d'évaluer ce travail, ainsi qu'à mes camarades de promotion pour l'esprit d'entraide et de partage qui a marqué notre parcours commun.

Que toutes celles et tous ceux que je n'ai pas pu citer nommément trouvent ici l'expression de ma sincère reconnaissance.

# Résumé

La gestion du portefeuille client constitue un enjeu stratégique majeur pour les cabinets de conseil financier. Au sein du cabinet AFFIC, cette gestion reposait sur des outils bureautiques dispersés (tableurs, messagerie électronique, dossiers partagés), engendrant des pertes d'information, des difficultés de suivi des tâches et de la facturation, ainsi que des risques importants en matière de confidentialité des données clients.

Ce mémoire présente la conception et la réalisation, au sein de la startup DataBeez, spécialisée en Data et Intelligence Artificielle, d'une plateforme web sécurisée de gestion de portefeuille client, dénommée AFFIC, intégrant un assistant conversationnel basé sur l'intelligence artificielle. La solution couvre la gestion des clients et de leurs portefeuilles, le suivi des tâches et des échéances, la facturation avec signature électronique, une DataRoom documentaire sécurisée, un système de notifications et un tableau de bord décisionnel enrichi d'un mécanisme de scoring client.

Une attention particulière a été portée à la sécurité de l'application, conformément aux référentiels OWASP : authentification forte à double facteur (code à usage unique envoyé par courriel), politique de mots de passe avec hachage salé, contrôle d'accès par profil avec vérification fine des ressources, validation systématique des entrées, chiffrement des échanges, accès temporaire et contrôlé aux documents, journalisation des événements de sécurité et traçabilité complète des actions utilisateurs.

La plateforme est déployée en production sur AWS. Sa sécurité a été vérifiée par un script de tests automatisés fondé sur l'OWASP, par les outils sqlmap et Burp Suite et par une analyse SonarQube. Certains points restent à valider, notamment deux tests non exécutés, l'évaluation de l'assistant IA et la mesure de la disponibilité et des temps de réponse.

Mots-clés : gestion de portefeuille client, sécurité des applications web, authentification forte, contrôle d'accès, OWASP, assistant IA, cloud.

---

# Abstract

Client portfolio management is a major strategic challenge for financial advisory firms. At AFFIC, this management relied on scattered office tools (spreadsheets, e-mail, shared folders), leading to

information loss, poor task and invoicing follow-up, and significant confidentiality risks regarding client data.

This thesis presents the design and implementation, within DataBeez, a startup specialising in Data and Artificial Intelligence, of a secure web platform for client portfolio management, called AFFIC, which integrates an AI-powered conversational assistant. The solution covers client and portfolio management, task and deadline tracking, invoicing with electronic signature, a secure document DataRoom, a notification system and a decision-making dashboard enhanced with a client scoring mechanism.

Particular attention was paid to application security, in line with OWASP guidelines : strong two-factor user authentication (one-time code sent by email), a password policy with salted hashing, profile-based access control with fine-grained resource checks, systematic input validation, encrypted communications, temporary and controlled access to documents, security event logging and full traceability of user actions.

The platform is deployed to production on AWS. Its security was verified with an automated OWASP-based test script, the sqlmap and Burp Suite tools and a SonarQube analysis. Some points remain to be validated, notably two unexecuted tests, the evaluation of the AI assistant and the measurement of availability and response times.

Keywords : client portfolio management, web application security, strong authentication, access control, OWASP, AI assistant, cloud.

# Table des matières

**Dédicace** i

**Remerciements** ii

**Résumé** iii

**Abstract** iii

**Liste des tableaux** viii

**Liste des figures** ix

**Liste des listings** x

**Liste des sigles et abréviations** xi

**Introduction générale** 1

**1 Présentation générale** 2

Introduction . . . 2

1.1 Présentation de la structure d'accueil . . . 2

1.1.1 DataBeez . . . 2

1.1.2 Le cabinet AFFIC, bénéficiaire du projet . . . 3

1.1.3 Organisation, acteurs et cycle de vie d'un mandat . . . 3

1.2 Présentation du sujet . . . 3

1.2.1 Contexte . . . 3

1.2.2 Problématique . . . 3

1.2.3 Objectifs . . . 4

1.2.4 Cadre d'étude et limites . . . 4

1.3 Démarche de conduite du projet . . . 5

1.3.1 Choix de la méthodologie . . . 5

1.3.2 Planning de réalisation . . . 5

Conclusion . . . 5

**2 Étude de l'existant, état de l'art et conception** **6**
Introduction . . . 6
2.1 Étude de l'existant . . . 6
2.1.1 Description de l'existant . . . 6
2.1.2 Critique de l'existant . . . 6
2.2 État de l'art . . . 6
2.2.1 Gestion de portefeuille client . . . 6
2.2.2 Solutions existantes et choix du développement sur mesure . . . 7
2.2.3 Sécurité des applications web . . . 7
2.2.4 Architectures web et intelligence artificielle . . . 8
2.3 Spécification des besoins . . . 8
2.3.1 Identification des acteurs . . . 8
2.3.2 Besoins fonctionnels . . . 9
2.3.3 Priorisation des besoins selon la méthode MoSCoW . . . 10
2.3.4 Besoins non fonctionnels . . . 10
2.3.5 Matrice des habilitations . . . 10
2.4 Modélisation UML . . . 12
2.4.1 Diagramme de cas d'utilisation global . . . 12
2.4.2 Description textuelle d'un cas d'utilisation . . . 12
2.5 Modélisation des données . . . 12
2.5.1 Modèle conceptuel de données (MCD) . . . 12
2.5.2 Modèle logique de données (MLD) . . . 14
Conclusion . . . 14

**3 Choix techniques, réalisation et déploiement** **15**
Introduction . . . 15
3.1 Architecture et choix des technologies . . . 15
3.1.1 Architecture logique . . . 15
3.1.2 Choix des technologies . . . 16
3.2 Choix du modèle de contrôle d'accès . . . 17
3.3 Politique d'authentification et de mots de passe . . . 18
3.3.1 Politique de mots de passe . . . 18
3.3.2 Hachage salé des mots de passe . . . 18
3.3.3 Jeton d'accès et gestion des sessions . . . 18
3.3.4 Double authentification par code à usage unique . . . 19
3.3.5 Couverture du risque OWASP A07:2025 (*Authentication Failures*) . . . 19
3.4 Réalisation . . . 20
3.4.1 Backend et frontend . . . 20
3.4.2 Présentation de l'application . . . 21

3.4.3 Scoring client . . . 22
3.4.4 Assistant IA . . . 22
3.5 Mesures de sécurité mises en œuvre . . . 23
3.6 Tests . . . 24
3.6.1 Tests de sécurité . . . 24
3.6.2 Analyse de la qualité du code avec SonarQube . . . 25
3.7 Déploiement en production . . . 27
3.8 Points non encore validés . . . 27
Conclusion . . . 28

**Conclusion générale et perspectives** **29**
Bilan du travail réalisé . . . 29
Apport personnel du projet . . . 29
Perspectives . . . 30

**RÉFÉRENCES BIBLIOGRAPHIQUES** **31**

**Annexes** **34**

**I Extrait du schéma de données (Prisma)** **34**

**II Principaux points de terminaison de l'API** **35**

**III Captures d'écran de la plateforme** **36**

**IV Extraits de code source** **37**

**V Captures des tests de sécurité** **40**

**VI Présentation de SonarQube** **44**

**VII Déploiement sur AWS** **46**

# Liste des tableaux


1.1 Comparaison des méthodologies de conduite de projet . . . . . 5

2.1 Insuffisances de l'existant et conséquences . . . . . 7
2.2 Comparaison des solutions existantes . . . . . 8
2.3 Acteurs du système . . . . . 9
2.4 Besoins fonctionnels par module . . . . . 9
2.5 Priorisation des besoins selon la méthode MoSCoW . . . . . 10
2.6 Besoins non fonctionnels . . . . . 11
2.7 Matrice des habilitations par profil . . . . . 11
2.8 Description du cas d'utilisation « Téléverser un document dans la DataRoom » . . . 13

3.1 Comparaison des technologies et choix retenus . . . . . 16
3.2 Comparaison des modèles de contrôle d'accès au regard des besoins d'AFFIC . . . . 17
3.3 Politique de mots de passe de la plateforme . . . . . 18
3.4 Mesures de protection de la double authentification . . . . . 19
3.5 Causes de défaillance de l'authentification (OWASP A07:2025) et prise en compte . 20
3.6 Contre-mesures implémentées par risque OWASP Top 10:2025 . . . . . 23
3.7 Cas de tests de sécurité et résultats obtenus . . . . . 24
3.8 Bogues détectés par SonarQube et corrections . . . . . 25
3.9 Revue des points sensibles de sécurité . . . . . 26
3.10 Résultats SonarQube avant et après correction . . . . . 26
3.11 Estimation des coûts mensuels de l'infrastructure AWS (1 € = 655,957 FCFA) . . . 27

II.1 Points de terminaison de l'API REST (extrait) . . . . . 35

# Liste des figures

1.1 Organigramme simplifié de DataBeez et position de l'auteur . . . . . 3

2.1 Diagramme de cas d'utilisation global du système AFFIC . . . . . 12
2.2 Modèle conceptuel de données (extrait) . . . . . 13

3.1 Architecture logique de la plateforme . . . . . 15
3.2 Chaîne de traitement sécurisée d'une requête API . . . . . 17
3.3 Connexion avec double authentification par code envoyé par courriel . . . . . 19
3.4 Tableau de bord de la plateforme AFFIC . . . . . 21
3.5 Gestion des clients . . . . . 21
3.6 Gestion des tâches . . . . . 21
3.7 DataRoom et gestion documentaire . . . . . 22
3.8 Assistant IA conversationnel . . . . . 22

III.1 Vue d'ensemble du portefeuille . . . . . 36
III.2 Module de facturation . . . . . 36

V.1 Test d'accès à l'API sans jeton : réponse 401 . . . . . 40
V.2 Test d'utilisation d'un jeton altéré : réponse 401 . . . . . 40
V.3 Test d'accès d'un collaborateur au client d'un autre : réponse 403 . . . . . 41
V.4 Test d'envoi d'une requête malformée : réponse 400 . . . . . 41
V.5 Test de tentative d'injection SQL : aucun effet . . . . . 41
V.6 Test d'échecs de connexion répétés : réponse 401 à chaque tentative . . . . . 42
V.7 Interception du trafic de la plateforme dans Burp Suite . . . . . 42
V.8 Modification d'une requête dans Burp Repeater : l'injection SQL reste sans effet . . 43

VI.1 Tableau de bord SonarQube lors de la première analyse (11 octobre 2026, code global) 45
VI.2 Tableau de bord SonarQube lors de la seconde analyse (12 octobre 2026, code modifié) 45

VII.1 Topologie de déploiement de la plateforme sur AWS . . . . . 46
VII.2 Tableau de bord du pare-feu applicatif AWS WAF associé à la distribution CloudFront 47

# Liste des listings


3.1 Hachage salé du mot de passe avec bcrypt . . . . . 18
I.1 Extrait du fichier schema.prisma . . . . . 34
IV.1 Génération du code et du jeton de défi (`twoFactorService.ts`, extrait) . . . . . 37
IV.2 Arborescence du backend . . . . . 37
IV.3 Middleware de contrôle d'accès aux clients (extrait) . . . . . 37
IV.4 Extrait du script de tests : contrôle d'accès au niveau de la ressource . . . . . 38
IV.5 Lancement de sqlmap sur la route de recherche des clients . . . . . 38
IV.6 Configuration de l'analyse SonarQube (`sonar-project.properties`) . . . . . 38
IV.7 Dockerfile du backend (extrait) . . . . . 38
IV.8 Script de déploiement automatisé du backend (extrait) . . . . . 39

# Liste des sigles et abréviations

| | |
|---|---|
| ABAC | Attribute-Based Access Control |
| ACID | Atomicité, Cohérence, Isolation, Durabilité |
| AI | Artificial Intelligence (intelligence artificielle, résumé anglais) |
| ALB | Application Load Balancer |
| ANSSI | Agence nationale de la sécurité des systèmes d'information |
| API | Application Programming Interface |
| AWS | Amazon Web Services |
| CAMES | Conseil Africain et Malgache pour l'Enseignement Supérieur |
| CDN | Content Delivery Network |
| CORS | Cross-Origin Resource Sharing |
| CRM | Customer Relationship Management |
| CRUD | Create, Read, Update, Delete |
| CSRF | Cross-Site Request Forgery |
| DAC | Discretionary Access Control |
| ECR | Elastic Container Registry |
| ECS | Elastic Container Service |
| ETI | Entreprise de Taille Intermédiaire |
| FCFA | Franc de la Communauté Financière Africaine |
| FK | Foreign Key (clé étrangère) |
| Go | Gigaoctet |
| GPT | Generative Pre-trained Transformer |
| HMAC | Hash-based Message Authentication Code |
| HT | Hors Taxes |
| HTTP(S) | HyperText Transfer Protocol (Secure) |

| | |
|---|---|
| IA | Intelligence Artificielle |
| IAM | Identity and Access Management |
| IDOR | Insecure Direct Object Reference (référence directe non sécurisée à un objet) |
| IEEE | Institute of Electrical and Electronics Engineers |
| IETF | Internet Engineering Task Force |
| IP | Internet Protocol |
| JSON | JavaScript Object Notation |
| JWT | JSON Web Token |
| LLM | Large Language Model (grand modèle de langage) |
| MAC | Mandatory Access Control |
| MCD | Modèle Conceptuel de Données |
| MD5 | Message Digest 5 (fonction de hachage obsolète) |
| MLD | Modèle Logique de Données |
| Mo | Mégaoctet |
| MoSCoW | Must have, Should have, Could have, Won't have |
| NIST | National Institute of Standards and Technology |
| NLP | Natural Language Processing (traitement automatique du langage naturel) |
| ORM | Object-Relational Mapping |
| OWASP | Open Worldwide Application Security Project |
| PDF | Portable Document Format |
| PME | Petites et Moyennes Entreprises |
| RAG | Retrieval-Augmented Generation |
| RAM | Random Access Memory (mémoire vive) |
| RBAC | Role-Based Access Control |
| RDS | Relational Database Service |
| ReDoS | Regular expression Denial of Service (déni de service par expression régulière) |
| REST | Representational State Transfer |
| RFC | Request For Comments |
| S3 | Simple Storage Service |

| | |
|---|---|
| SAS | Société par Actions Simplifiée |
| SDK | Software Development Kit |
| SGBD | Système de Gestion de Base de Données |
| SHA-256 | Secure Hash Algorithm, 256 bits |
| SPA | Single Page Application |
| SQL | Structured Query Language |
| TLS | Transport Layer Security |
| TOTP | Time-based One-Time Password |
| TVA | Taxe sur la Valeur Ajoutée |
| UML | Unified Modeling Language |
| URL | Uniform Resource Locator |
| USB | Universal Serial Bus |
| vCPU | virtual Central Processing Unit (processeur virtuel) |
| WAF | Web Application Firewall (pare-feu applicatif web) |
| XSS | Cross-Site Scripting |

# Introduction générale

La transformation numérique bouleverse le secteur du conseil financier, où les cabinets manipulent un volume croissant d'informations sensibles : identité des clients, montants, documents contractuels, mandats confidentiels. Au Sénégal, cette digitalisation s'accompagne d'exigences réglementaires renforcées, notamment la loi n$^{o}$ 2008-12 du 25 janvier 2008 sur la protection des données à caractère personnel [18], qui imposent de concilier ouverture des systèmes d'information et maîtrise de la sécurité.

C'est dans ce contexte que s'inscrit le présent travail, réalisé au sein de DataBeez, startup dakaroise spécialisée dans la formation, le conseil et l'ingénierie de la donnée et de l'intelligence artificielle, pour le compte de son client AFFIC, cabinet de conseil financier basé à Dakar, qui souhaitait centraliser et sécuriser la gestion de son portefeuille client, c'est-à-dire le suivi de l'ensemble de ses clients, de leurs mandats et des documents associés.

Ce constat soulève la question suivante : comment concevoir et mettre en œuvre une plateforme de gestion de portefeuille client qui centralise l'ensemble des processus métier du cabinet, à savoir les clients, les tâches, les documents, la facturation et le pilotage, tout en garantissant un niveau de sécurité conforme aux exigences de confidentialité du secteur financier ?

Pour y répondre, ce travail vise à concevoir, développer, sécuriser et déployer une application web de gestion de portefeuille client, enrichie d'un assistant conversationnel fondé sur l'intelligence artificielle et protégée par une politique de sécurité couvrant l'authentification, les habilitations, le chiffrement et la traçabilité des actions.

Le présent mémoire est organisé en trois chapitres. Le premier chapitre présente le cadre général du projet : la structure d'accueil, le sujet, ses objectifs et la démarche de conduite du projet. Ensuite, le deuxième chapitre est consacré à l'étude de l'existant, à l'état de l'art, puis à l'analyse des besoins et à la conception générale de la solution. Enfin, le troisième chapitre expose nos choix techniques et leur justification, puis décrit la réalisation, la sécurisation, les tests et le déploiement de la plateforme. Une conclusion générale dresse le bilan du travail, présente notre apport personnel et ouvre sur les perspectives.

# Chapitre 1

# Présentation générale

## Introduction

Ce chapitre présente la structure d'accueil et le cabinet AFFIC, puis le sujet (contexte, problématique, objectifs) et la démarche de conduite du projet.

## 1.1 Présentation de la structure d'accueil

### 1.1.1 DataBeez

Le présent travail a été réalisé au sein de DataBeez (Databeez SAS), société par actions simplifiée créée le 31 août 2021 et installée à l'Immeuble Arcanes, Point E, Dakar.

DataBeez couvre la formation en Data et IA, le conseil en systèmes d'information, l'ingénierie et la valorisation des données, cadre du présent projet, et le recrutement de talents spécialisés.

Cette jeune entreprise (*startup*) s'est donné pour mission de « mettre la donnée et l'intelligence artificielle au service de l'impact, de la souveraineté numérique africaine et de l'innovation inclusive ». En quelques années, elle a accompagné des organisations de référence (Sonatel, RYSAMM Holding, l'École Supérieure Polytechnique de Dakar) à travers des formations et des projets de conseil en données et en IA.

L'organigramme simplifié de la figure 1.1 situe l'auteur au sein de la structure : le projet AFFIC a été mené dans le pôle Conseil et ingénierie Data/IA, où nous occupions le poste de développeur.

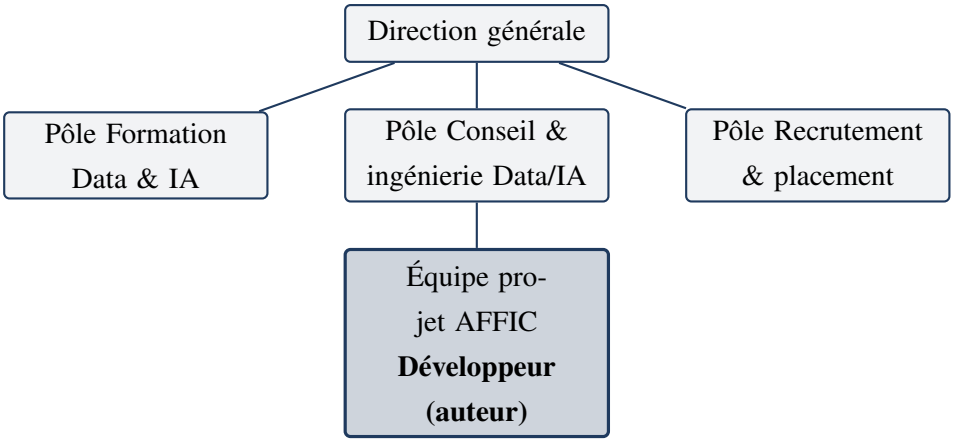



FIGURE 1.1 – Organigramme simplifié de DataBeez et position de l'auteur

### 1.1.2 Le cabinet AFFIC, bénéficiaire du projet

AFFIC est un cabinet de conseil financier basé à Dakar, qui accompagne des PME et des ETI de la sous-région dans leurs opérations de financement et de structuration (levées de fonds, dettes structurées, fusions-acquisitions), en tant qu'intermédiaire de confiance avec des partenaires financiers.

### 1.1.3 Organisation, acteurs et cycle de vie d'un mandat

L'activité du cabinet mobilise quatre profils : la direction, les managers de portefeuilles, les collaborateurs affectés à des clients et des profils en lecture seule (contrôle interne, partenaires). Un mandat, mission confiée par un client, suit un cycle type : prospection ; signature, qui arrête les honoraires fixes (*retainer*) et la commission de succès (*success fee*) ; exécution (*due diligence*, *roadshow* auprès des investisseurs, négociation, *closing*) ; enfin facturation et clôture.

## 1.2 Présentation du sujet

### 1.2.1 Contexte

Les cabinets de conseil financier manipulent au quotidien des informations hautement confidentielles (états financiers, mandats, *term sheets*, données personnelles des dirigeants) et doivent suivre précisément chaque client, mandat et échéance. Or, chez AFFIC, ce suivi reposait sur des outils bureautiques généralistes (section 2.1) ; DataBeez a donc été sollicitée pour concevoir une plateforme dédiée.

### 1.2.2 Problématique

Ces insuffisances touchent l'efficacité du cabinet et sa sécurité. La problématique se décline donc en deux questions : comment réunir dans un outil unique le référentiel client, les mandats, les tâches,

les documents et la facturation ? Et comment garantir que chaque utilisateur n'accède qu'aux informations qui le concernent, avec une trace de chacune de ses actions ?

### 1.2.3 Objectifs

#### Objectif principal

L'objectif principal est de concevoir, développer, sécuriser et déployer une plateforme web de gestion de portefeuille client couvrant l'ensemble des processus métier du cabinet AFFIC, et intégrant un assistant conversationnel fondé sur l'intelligence artificielle.

#### Objectifs spécifiques

Il se décline en objectifs spécifiques :

1. mettre en place un référentiel client unique (historique, notes, scores, documents, tâches liées) ;
2. gérer le portefeuille de mandats (montants, honoraires, étapes, avancement) ;
3. structurer la gestion des tâches (affectation, priorités, échéances, notifications, vues liste, kanban et calendrier) ;
4. automatiser la facturation (numérotation, TVA de 18 %, suivi des paiements, signature électronique) ;
5. mettre en place une DataRoom sécurisée, « salle de données » virtuelle où les documents confidentiels ne sont partagés qu'avec les personnes habilitées ;
6. offrir un tableau de bord décisionnel (indicateurs, segmentation, scoring client) ;
7. intégrer un assistant conversationnel respectueux des habilitations ;
8. garantir la sécurité de bout en bout : authentification forte, contrôle des habilitations, validation des entrées, chiffrement et journalisation.

### 1.2.4 Cadre d'étude et limites

Le périmètre de ce travail est volontairement circonscrit, ce qui en délimite aussi la portée. L'étude a été menée auprès d'un *seul* cabinet, AFFIC, dont les besoins ont servi de référence : les choix fonctionnels et les règles métier retenus reflètent son organisation et ne prétendent pas couvrir l'ensemble des cabinets de conseil financier. La généralisation à d'autres structures nécessiterait une phase d'adaptation.

Par ailleurs, la réalisation a reposé sur un *unique* développeur, qui a cumulé la conception, le développement, les tests et le déploiement. Cette configuration a favorisé la cohérence des choix techniques, mais elle a limité le volume de fonctionnalités livrables dans le temps imparti et n'a pas permis la revue de code par un pair. Les perspectives d'évolution présentées en conclusion tiennent compte de ces deux limites.

## 1.3 Démarche de conduite du projet

### 1.3.1 Choix de la méthodologie

Plusieurs démarches de conduite de projet ont été envisagées. Le tableau 1.1 en présente une comparaison synthétique.

TABLEAU 1.1 – Comparaison des méthodologies de conduite de projet

| **Méthode** | **Atouts** | **Limites dans notre contexte** |
|---|---|---|
| Cycle en V | Cadre rigoureux, phases bien définies | Rigide face à des besoins qui s'affinent en cours de projet |
| 2TUP (*Two Track Unified Process*) | Séparation des branches fonctionnelle et technique | Lourd pour une équipe réduite |
| Scrum | Livraisons incrémentales, rétroaction courte, priorisation par la valeur | Exige une forte implication du client |

Comme le montre le tableau 1.1, nous avons retenu Scrum [16], car les besoins devaient s'affiner en cours de projet et les fonctionnalités à forte valeur être livrées rapidement. Un représentant de la direction d'AFFIC tenait le rôle de *Product Owner* ; nous assurions ceux de *Scrum Master* et de développeur. Le travail était découpé en *sprints* de deux semaines, conclus par une revue avec le client et une rétrospective, à partir d'un *backlog* de récits utilisateurs priorisés par valeur et par risque. L'analyse s'est appuyée sur UML pour les fonctionnalités et sur Merise pour les données. Selon le principe de sécurité dès la conception (*security by design*), chaque récit utilisateur intégrait des critères d'acceptation de sécurité, revus en fin de sprint.

### 1.3.2 Planning de réalisation

Le projet s'est déroulé sur vingt-quatre semaines, de mai à octobre 2026, en quatre phases : cadrage, analyse des besoins et architecture (3 semaines) ; modules cœur, à savoir authentification, clients, tâches et documents (11 semaines) ; facturation, tableau de bord, scoring et assistant IA (6 semaines) ; enfin tests, durcissement de la sécurité et déploiement (4 semaines).

## Conclusion

Ce chapitre a posé le cadre du projet : environnement, problématique, objectifs et démarche agile. Le chapitre suivant analyse l'existant et les besoins.

# Chapitre 2

# Étude de l'existant, état de l'art et conception

## Introduction

Ce chapitre présente l'existant, un état de l'art, puis les besoins et la conception de la solution, indépendamment des outils retenus.

## 2.1 Étude de l'existant

### 2.1.1 Description de l'existant

Avant le projet, le système d'information du cabinet reposait sur des classeurs Excel (clients, mandats, factures), la messagerie électronique (échanges, affectation informelle des tâches, transmission de documents), des dossiers partagés et des documents Word pour les livrables. Cette organisation a été analysée au moyen d'entretiens avec les utilisateurs.

### 2.1.2 Critique de l'existant

L'analyse de l'existant a mis en évidence des insuffisances majeures, récapitulées dans le tableau 2.1.

Parmi ces insuffisances, les risques de sécurité constituent le point le plus critique, car ils exposent le cabinet à des fuites de données et à une non-conformité à la réglementation sénégalaise sur la protection des données à caractère personnel [18].

## 2.2 État de l'art

### 2.2.1 Gestion de portefeuille client

La gestion de la relation client (CRM) vise à capter et exploiter les informations clients pour optimiser la relation commerciale [2] ; intégrée au système d'information de l'organisation [1], elle

TABLEAU 2.1 – Insuffisances de l'existant et conséquences

| **Insuffisance** | **Conséquences** |
|---|---|
| Dispersion de l'information | Données dupliquées dans plusieurs fichiers sans référentiel unique, mises à jour non synchronisées, versions contradictoires. |
| Absence de pilotage structuré | Suivi des tâches et des échéances reposant sur la mémoire des collaborateurs ; aucune vue consolidée de la charge de travail ni de l'avancement des mandats. |
| Facturation manuelle | Préparation sous tableur, sans numérotation automatique fiable ni suivi systématique des paiements. |
| Risques de sécurité | Documents confidentiels circulant par courriel ou clé USB, sans chiffrement, sans contrôle des habilitations ni traçabilité des accès ; absence d'authentification centralisée et de journal d'audit. |

s'enrichit, dans le conseil financier, du suivi de mandats (montants, honoraires, étapes, facturation). Un tel système doit couvrir le référentiel client, le cycle de vie des mandats, les tâches, une gestion documentaire sécurisée (DataRoom), la facturation et le pilotage, notamment par un scoring client : une note combinant revenu, activité et potentiel, dont la formule et l'état actuel sont précisés à la section 3.4.3.

### 2.2.2 Solutions existantes et choix du développement sur mesure

Plusieurs solutions du marché ont été étudiées. Le tableau 2.2 en présente les points forts et les limites au regard des besoins du cabinet.

Aucune solution ne couvre, à un coût soutenable, les mandats et leurs honoraires, la TVA sénégalaise de 18 %, une DataRoom à droits granulaires et un assistant IA soumis aux habilitations : un développement sur mesure a donc été retenu.

### 2.2.3 Sécurité des applications web

La sécurité d'un système d'information repose sur la disponibilité, l'intégrité et la confidentialité, complétées par la traçabilité [6]. Le Top 10:2025 de l'OWASP [3], dernière édition, classe les risques les plus critiques des applications web ; ceux qui concernent notre plateforme sont A01 (contrôle d'accès), A02 (configuration), A03 (chaîne d'approvisionnement logicielle), A04 (cryptographie), A05 (injection), A06 (conception), A07 (authentification), A08 (intégrité), A09 (journalisation et alertes) et A10 (gestion des conditions exceptionnelles). Par rapport à l'édition 2021, la configuration passe de A05 à A02, la cryptographie de A02 à A04, l'injection de A03 à A05 et la conception de A04 à A06. Cette liste sert de grille d'analyse tout au long du projet.

L'authentification forte combine une politique de mots de passe, un stockage sous forme d'empreinte salée, la limitation des tentatives, des sessions de durée limitée et, idéalement, plusieurs fac-

TABLEAU 2.2 – Comparaison des solutions existantes

| Solution | Points forts | Limites pour AFFIC |
|---|---|---|
| Salesforce [28] | CRM leader, très complet | Coût élevé, complexité et fonctionnalités surdimensionnées pour un cabinet de petite taille ; faible maîtrise de la configuration et pas de DataRoom ni de TVA locale intégrées |
| Microsoft Dynamics 365 [29] | Intégration Office, modules finance | Coût, lourdeur de déploiement |
| HubSpot [30] | Prise en main aisée | Orienté marketing ; DataRoom et TVA locale non couvertes |
| Odoo [31] | Open source, modulaire | Personnalisation lourde pour le métier du conseil |
| Intralinks [32], Drooms [33] | Partage documentaire sécurisé | Volet documentaire seul, coût élevé |

teurs [7]. Le contrôle d'accès repose sur l'un des quatre modèles classiques [10] : discrétionnaire (DAC), obligatoire (MAC), à base de rôles (RBAC) [11] ou à base d'attributs (ABAC) [12], comparés au chapitre 3. S'y ajoutent la validation des entrées, les requêtes paramétrées contre les injections, le chiffrement des échanges et des données, et la journalisation des événements.

### 2.2.4 Architectures web et intelligence artificielle

Une application monopage (SPA) consomme une API REST [13], sans état, qui expose les ressources du métier (clients, tâches, factures). Les grands modèles de langage (LLM) permettent par ailleurs d'interroger ces données en langage naturel ; la génération augmentée par la recherche (RAG) [14] ancre leurs réponses dans les documents de l'organisation et limite les hallucinations. L'OWASP recommande d'appliquer les filtres d'autorisation en amont de la génération, plutôt que de les confier au modèle [5].

## 2.3 Spécification des besoins

### 2.3.1 Identification des acteurs

Quatre profils d'utilisateurs interagissent avec la plateforme. Ils sont décrits dans le tableau 2.3.

À ces acteurs humains s'ajoutent des acteurs systèmes : un service de stockage de fichiers, un service d'envoi de courriels pour les notifications et un moteur d'intelligence artificielle pour l'assistant conversationnel.

TABLEAU 2.3 – Acteurs du système

| Acteur | Description |
|---|---|
| Administrateur | Gère les utilisateurs et leurs profils, supervise la sécurité (journal des événements, sessions) et dispose de tous les droits. |
| Manager | Pilote l'ensemble des clients et des mandats : création de clients, affectation des responsables, gestion des tâches, facturation, accès à tous les documents. |
| Collaborateur | Travaille sur les clients qui lui sont assignés : tâches, documents et événements liés à son périmètre uniquement. |
| Lecture seule | Consulte, sans pouvoir les modifier, les informations de son périmètre (clients, tâches, factures, documents). |

### 2.3.2 Besoins fonctionnels

Les besoins fonctionnels, recueillis lors d'ateliers avec les utilisateurs, se répartissent en dix modules (tableau 2.4) ; compte tenu de la sensibilité des données, la sécurité y est traitée comme un module à part entière.

TABLEAU 2.4 – Besoins fonctionnels par module

| Module | Fonctionnalités |
|---|---|
| Sécurité | Authentification forte ; politique de mots de passe ; gestion et révocation des sessions ; contrôle d'accès selon le profil et le périmètre de l'utilisateur ; journal des événements de sécurité ; traçabilité des actions. |
| Profil | Connexion et déconnexion ; gestion du profil (photo, signature). |
| Gestion des clients | Création, consultation, modification et suppression des clients et prospects ; affectation d'un responsable ; fiche à onglets (historique, notes, tâches, documents) ; tri alphabétique ou par date ; scores (revenu, activité, potentiel). |
| Portefeuille | Secteur, projet, montant, honoraires (retainer, success fee), durée, dates de closing, avancement par étapes. |
| Gestion des tâches | Création et suivi des tâches, affectation avec notification par courriel, priorités, statuts et échéances, vues liste, kanban et calendrier, métriques. |
| Facturation | Factures multi-articles, numérotation automatique, TVA de 18 %, suivi des statuts, signature électronique, export PDF. |

| Module | Fonctionnalités |
|---|---|
| DataRoom | Arborescence de dossiers, téléversement, prévisualisation PDF, Word, Excel et PowerPoint, renommage, partage à niveaux d'accès, extraction de texte. |
| Agenda | Calendrier des événements (réunions, échéances) avec assignation et rattachement à un client. |
| Tableau de bord | Indicateurs clés, graphiques de performance, segmentation, échéances à venir, scoring. |
| Assistant IA | Interrogation en langage naturel, historique, suggestions, respect strict des habilitations. |

### 2.3.3 Priorisation des besoins selon la méthode MoSCoW

Le backlog a été priorisé selon la méthode MoSCoW (*Must*, *Should*, *Could*, *Won't have*), comme le présente le tableau 2.5.

TABLEAU 2.5 – Priorisation des besoins selon la méthode MoSCoW

| Priorité | Besoins |
|---|---|
| Must have | S'authentifier ; contrôle d'accès selon le profil ; gérer les clients ; gérer les tâches ; gérer la DataRoom ; journaliser les actions et les événements de sécurité. |
| Should have | Gérer la facturation ; gérer le portefeuille de mandats ; notifier par courriel l'utilisateur assigné ; consulter le tableau de bord ; second facteur d'authentification par code envoyé par courriel. |
| Could have | Signer électroniquement une facture ; interroger l'assistant IA ; agenda partagé ; scoring client. |
| Won't have | Application mobile native ; intégrations bancaires. |

### 2.3.4 Besoins non fonctionnels

Les exigences de qualité attendues de la plateforme sont résumées dans le tableau 2.6.

### 2.3.5 Matrice des habilitations

Les droits attendus de chaque profil sur les principales ressources sont formalisés dans le tableau 2.7 (C : créer, R : consulter, U : modifier, D : supprimer).

L'habilitation dépend donc du profil et du périmètre : un collaborateur n'agit que sur les clients qui lui sont assignés, et cette règle s'applique aussi aux factures pour le collaborateur et le profil lecture seule.

TABLEAU 2.6 – Besoins non fonctionnels

| **Exigence** | **Description** |
|---|---|
| Confidentialité | Chiffrement des échanges et des documents stockés ; aucun document accessible publiquement. |
| Performance | Objectif de temps de réponse inférieur à 3 secondes pour les écrans courants ; pagination des listes. |
| Disponibilité | Objectif de 99 % en heures ouvrées, compatible avec une instance unique ; sauvegardes automatiques de la base. |
| Ergonomie | Interface responsive, intuitive et en français. |
| Maintenabilité | Code typé ; séparation des couches ; API documentée. |
| Évolutivité | Architecture ouverte à de nouveaux modules sans refonte. |

TABLEAU 2.7 – Matrice des habilitations par profil

| **Ressource** | **Admin** | **Manager** | **Collaborateur** | **Lecture seule** |
|---|---|---|---|---|
| Gestion des utilisateurs | CRUD | — | — | — |
| Clients (tous / assignés) | CRUD | CRUD | — / CRUD | R |
| Tâches (toutes / assignées) | CRUD | CRUD | — / CRUD | R |
| Facturation (toutes / assignées) | CRUD | CRUD | R (assignées) | R (assignées) |
| Documents (globaux / clients) | CRUD | CRUD | CRUD / CRUD* | R |
| Journal de sécurité | R | — | — | — |
| Assistant IA | ✓ | ✓ | ✓ | ✓ |

* uniquement les documents des clients qui lui sont assignés.

## 2.4 Modélisation UML

### 2.4.1 Diagramme de cas d'utilisation global

La figure 2.1 présente le diagramme de cas d'utilisation global. Les profils sont reliés par généralisation : le collaborateur hérite des cas du profil lecture seule, le manager de ceux du collaborateur et l'administrateur de ceux du manager. « Gérer les tâches » inclut « Notifier l'utilisateur assigné », et « Signer électroniquement une facture » étend « Gérer la facturation ». L'authentification, préalable commun à tous les cas, est traitée comme une précondition et n'est pas représentée.

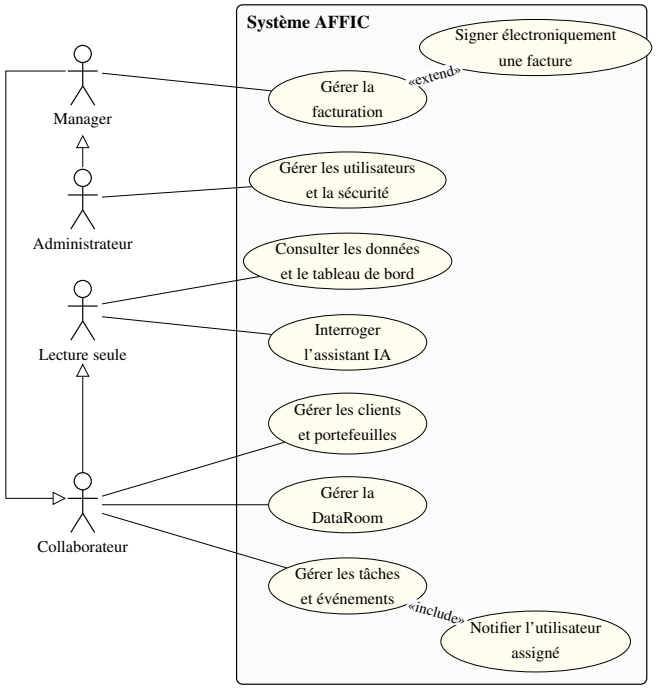



FIGURE 2.1 – Diagramme de cas d'utilisation global du système AFFIC

### 2.4.2 Description textuelle d'un cas d'utilisation

Le tableau 2.8 détaille un cas représentatif des contrôles attendus sur les ressources sensibles.

## 2.5 Modélisation des données

### 2.5.1 Modèle conceptuel de données (MCD)

Le modèle conceptuel de données compte une quinzaine d'entités ; la figure 2.2 présente les principales, avec leurs cardinalités.

TABLEAU 2.8 – Description du cas d'utilisation « Téléverser un document dans la DataRoom »

| **Rubrique** | **Description** |
|---|---|
| Acteurs | Manager, Collaborateur, Administrateur |
| Préconditions | Utilisateur authentifié ; droits d'écriture sur le dossier ou le client cible |
| Scénario nominal | 1. L'utilisateur sélectionne un dossier et dépose un fichier. 2. Le système vérifie l'authentification, le profil et l'accès au client concerné. 3. Le système valide le type et la taille du fichier. 4. Le fichier est transmis au service de stockage et une adresse d'accès sécurisée est générée. 5. Les métadonnées sont enregistrées. 6. L'action est journalisée. |
| Scénarios alternatifs | Droits insuffisants : accès refusé. Fichier non conforme : rejet avec message explicite. |
| Postconditions | Document disponible, tracé et soumis au contrôle d'accès. |

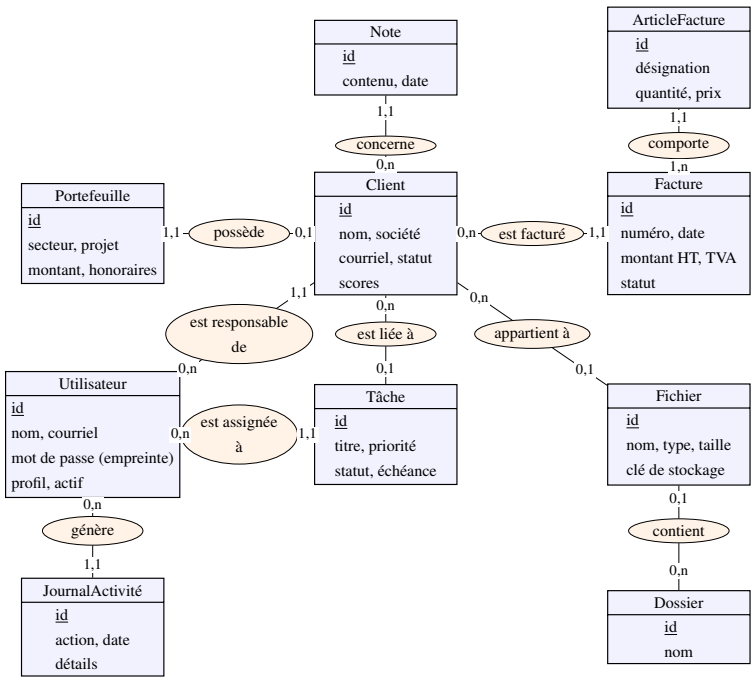



FIGURE 2.2 – Modèle conceptuel de données (extrait)

Les principales règles de gestion sont les suivantes : chaque client a un responsable unique et au plus un portefeuille de mandats ; un client peut avoir plusieurs notes, factures, tâches et événements ; une facture comporte au moins un article ; un dossier contient des fichiers et d'autres dossiers (association réflexive) ; un fichier ou un dossier peut être partagé avec plusieurs utilisateurs, l'association portant le niveau d'accès ; enfin, chaque action d'un utilisateur génère une entrée de journal.

### 2.5.2 Modèle logique de données (MLD)

Le modèle logique est obtenu par les règles de passage de Merise : une association un-à-plusieurs devient une clé étrangère, une association plusieurs-à-plusieurs ou porteuse d'attributs devient une table. Clé primaire soulignée, clés étrangères précédées de #, on obtient par exemple (schéma complet en annexe I) :

— Client (id, nom, société, courriel, statut, typeClient, valeurCompte, #responsableId, #crééParId) ;

— AccèsFichier (#fichierId, #utilisateurId, niveauAccès).

## Conclusion

Besoins, habilitations et données étant modélisés, le chapitre suivant présente notre réponse technique.

# Chapitre 3

# Choix techniques, réalisation et déploiement

## Introduction

Ce chapitre justifie nos choix (architecture, technologies, contrôle d'accès, authentification), puis décrit la réalisation, la sécurité, les tests, le déploiement et les points restant à valider.

## 3.1 Architecture et choix des technologies

### 3.1.1 Architecture logique

La solution suit une architecture trois tiers en couches [17], présentée à la figure 3.1 indépendamment des outils qui la mettent en œuvre.

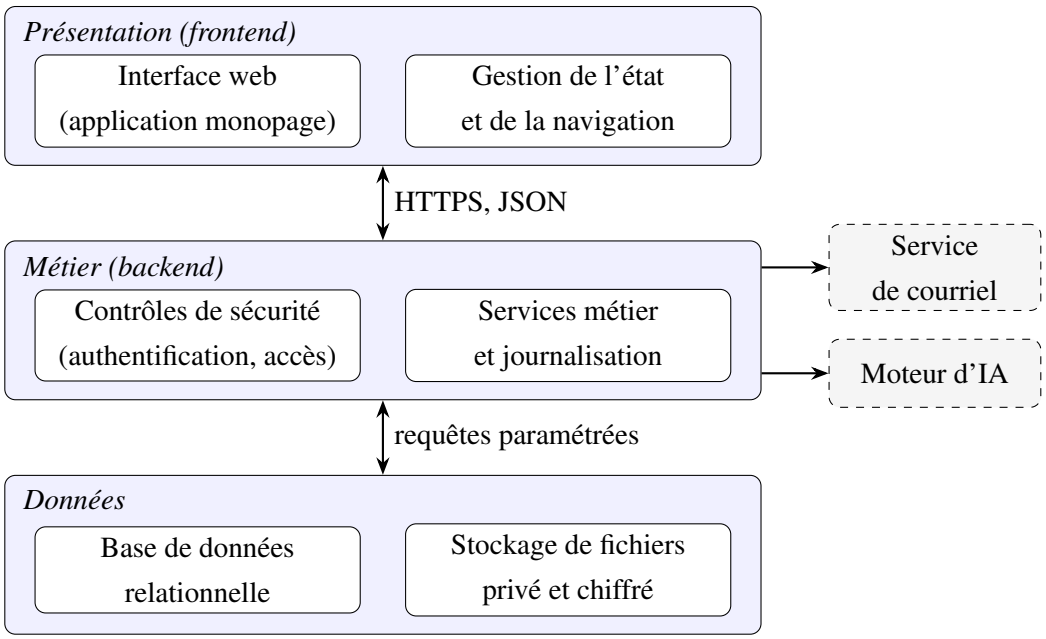



FIGURE 3.1 – Architecture logique de la plateforme

Le frontend ne fait qu'afficher et ne porte aucune règle de sécurité dont dépendrait la protection des

données. Chaque requête adressée à l'API REST du backend franchit d'abord les contrôles de sécurité (authentification, contrôle d'accès, validation des entrées) avant d'atteindre les services métier, seuls à accéder aux données et à journaliser les actions. Les services externes ne sont appelés que par le backend, ce qui concentre les contrôles en un point unique.

### 3.1.2 Choix des technologies

Le tableau 3.1 résume, pour chaque couche, les alternatives étudiées et la solution retenue.

TABLEAU 3.1 – Comparaison des technologies et choix retenus

| **Besoin** | **Alternatives** | **Choix retenu** | **Justification** |
|---|---|---|---|
| Frontend | Angular, Vue.js, React | React 18 + TypeScript [19] | Écosystème mature, typage statique ; plus léger qu'Angular pour une petite équipe. |
| Backend | Django, Spring Boot, Express | Node.js + Express [21] | Même langage que le frontend ; nombreux middlewares de sécurité. |
| SGBD | MySQL, MongoDB, PostgreSQL | PostgreSQL [22] | Transactions ACID, intégrité référentielle ; données fortement relationnelles. |
| Accès aux données | SQL brut, TypeORM, Prisma | Prisma [20] | Requêtes paramétrées par construction, client typé, migrations versionnées. |
| Validation | Joi, express-validator, Zod | Zod [24] | Schémas déclaratifs typés. |
| Authentification | Sessions serveur, JWT | JWT [8] | API sans état adaptée à une SPA ; révocation par re-vérification du compte à chaque requête. |
| Hachage | SHA-256, Argon2, bcrypt | bcrypt [9] | Fonction lente et salée, éprouvée. |

| Besoin | Alternatives | Choix retenu | Justification |
|---|---|---|---|
| Cloud | Azure, Google Cloud, AWS | AWS [23] | Services managés couvrant tous les besoins, gestion fine des droits (IAM). |

## 3.2 Choix du modèle de contrôle d'accès

Le tableau 3.2 confronte les quatre modèles de la littérature [10, 11, 12] aux droits formalisés dans le tableau 2.7.

TABLEAU 3.2 – Comparaison des modèles de contrôle d'accès au regard des besoins d'AFFIC

| Modèle | Principe | Adéquation au contexte |
|---|---|---|
| DAC | Le propriétaire de la ressource attribue lui-même les droits. | Pas de politique centrale ; risque de partages excessifs. |
| MAC | Droits imposés par des niveaux de classification. | Trop rigide : les droits suivent l'organisation, non des niveaux de secret. |
| RBAC | Permissions attachées à des rôles attribués aux utilisateurs. | Correspond aux quatre profils du cabinet ; administration simple ; moindre privilège et audit facilités. |
| ABAC | Décision calculée à partir d'attributs et du contexte. | Très expressif mais complexe à concevoir et à auditer pour une équipe réduite. |

Nous avons retenu RBAC, avec quatre rôles (ADMIN, MANAGER, COLLABORATOR, READONLY) : les profils du cabinet sont stables, l'arrivée ou le départ d'un collaborateur se limite à l'affectation d'un rôle, et le moindre privilège est facile à vérifier. RBAC seul ne sait cependant pas exprimer qu'un collaborateur n'agit que sur ses clients : nous l'avons complété par un contrôle au niveau de la ressource (responsable du client, assignataire de la tâche, bénéficiaire d'un partage), qui emprunte à ABAC un attribut de la ressource et à DAC le partage nominatif des documents. La figure 3.2 montre l'ordre des contrôles appliqués à chaque requête, qui répond au risque OWASP A01.

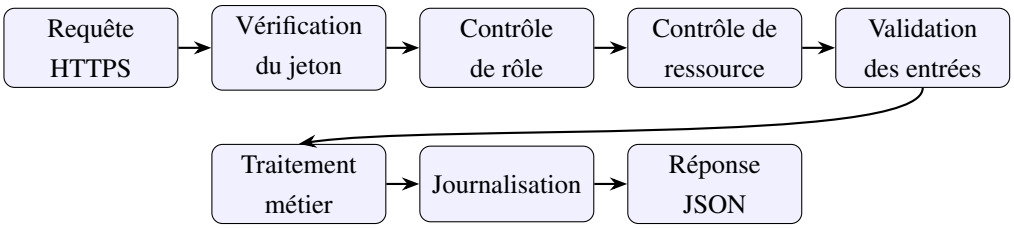



FIGURE 3.2 – Chaîne de traitement sécurisée d'une requête API

## 3.3 Politique d'authentification et de mots de passe

### 3.3.1 Politique de mots de passe

La politique retenue est résumée dans le tableau 3.3.

TABLEAU 3.3 – Politique de mots de passe de la plateforme

| **Règle** | **Mise en œuvre** |
|---|---|
| Création des comptes | Par un administrateur uniquement ; aucune inscription publique. |
| Longueur minimale | Huit caractères, contrôlés côté serveur par Zod, conformément au NIST [7] ; douze recommandés pour les comptes sensibles. |
| Stockage | Jamais en clair : seule l'empreinte bcrypt salée (facteur de coût 12) est conservée. |
| Changement | Par l'utilisateur ; réinitialisation possible par l'administrateur. |
| Messages d'erreur | Message générique unique, que le compte soit inexistant, désactivé ou le mot de passe erroné, contre l'énumération des comptes. |
| Désactivation | Perte immédiate de l'accès, même si le jeton n'a pas expiré. |

### 3.3.2 Hachage salé des mots de passe

Un sel est une valeur aléatoire ajoutée au mot de passe avant le hachage, afin que deux mots de passe identiques produisent des empreintes différentes ; il rend inefficaces les tables précalculées. Avec bcrypt [9], un sel de 128 bits est tiré à chaque appel et intégré à l'empreinte (`$2b$12$`, 22 caractères de sel, 31 de hachage), puis relu lors de la connexion (listing 3.1). Volontairement lent, bcrypt demande environ une centaine de millisecondes par empreinte avec un facteur de 12 : imperceptible pour l'utilisateur, rédhibitoire pour une attaque par force brute.

Listing 3.1 – Hachage salé du mot de passe avec bcrypt

```
1 const saltRounds = 12;   // facteur de cout : 2^12 iterations
2 const hashedPassword = await bcrypt.hash(password, saltRounds);
3 const isPasswordValid = await bcrypt.compare(password, user.password);
```

### 3.3.3 Jeton d'accès et gestion des sessions

Après authentification, le serveur délivre un jeton JWT signé (HMAC-SHA256), à durée de validité limitée, qui porte l'identité et le rôle de l'utilisateur. À chaque requête, l'existence et l'activité du compte sont re-vérifiées en base, ce qui permet de révoquer immédiatement un compte désactivé. Côté client, le jeton est conservé en mémoire vive, et non dans le `localStorage` ni dans un cookie : un script injecté (XSS) ne peut pas le lire de façon persistante, et comme il voyage dans l'en-tête

`Authorization`, une requête forgée depuis un site tiers (CSRF) ne peut pas l'emporter. Les en-têtes de sécurité (Helmet) et un paramétrage CORS restrictif complètent ces protections.

### 3.3.4 Double authentification par code à usage unique

Pour réduire le risque d'un mot de passe deviné ou dérobé (OWASP A07), la connexion exige un second facteur : un code à six chiffres envoyé par courriel (service Brevo). Si le mot de passe est correct, le serveur ne délivre pas encore le jeton d'accès mais un *jeton de défi* ; le jeton JWT n'est délivré qu'après vérification du code (figure 3.3).

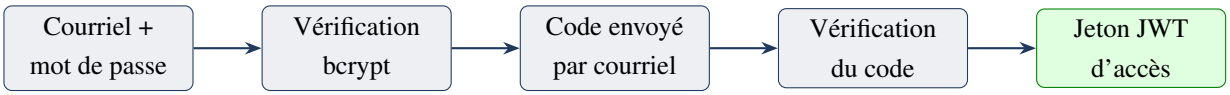



FIGURE 3.3 – Connexion avec double authentification par code envoyé par courriel

Le code est tiré par le générateur cryptographique `crypto.randomInt` et n'est jamais stocké en clair : seule son empreinte HMAC-SHA256 est placée dans le jeton de défi signé, valable dix minutes (listing IV.1, annexe IV). Les autres mesures sont résumées dans le tableau 3.4.

TABLEAU 3.4 – Mesures de protection de la double authentification

| **Mesure** | **Menace couverte** |
|---|---|
| Validité de 10 minutes, usage unique, ancien code invalidé au renvoi | Réutilisation ou rejeu d'un code |
| 5 essais par code, puis retour obligatoire à l'étape du mot de passe | Essai systématique des 1 000 000 de codes |
| 20 vérifications par quart d'heure et par adresse IP | Attaque automatisée sur la route |
| Comparaison en temps constant (`timingSafeEqual`) | Attaque temporelle |
| Jeton de défi refusé comme jeton d'accès | Contournement du second facteur |

Treize vérifications exécutées sur le service ont contrôlé cette logique ; l'utilisateur ne voit que le message générique « Une erreur est survenue ». Le courriel a été préféré à une application TOTP pour ne rien imposer à installer ; il est en revanche moins robuste, sa sécurité dépendant de celle de la boîte de courriel. Les cinq comptes de démonstration en sont dispensés (section 3.8).

### 3.3.5 Couverture du risque OWASP A07:2025 (*Authentication Failures*)

Le tableau 3.5 confronte les principales causes de défaillance de l'authentification recensées par l'OWASP Top 10:2025 [4] aux mesures de la plateforme.

TABLEAU 3.5 – Causes de défaillance de l'authentification (OWASP A07:2025) et prise en compte

| **Cause** | **Mesure dans AFFIC** | **Statut** |
|---|---|---|
| Mots de passe faibles ou par défaut | Huit caractères minimum contrôlés par Zod ; comptes créés par l'administrateur, sans inscription publique ni identifiant par défaut de type `admin/admin` en production. | Couvert |
| Absence de double authentification | Code à usage unique envoyé par courriel à chaque connexion (section 3.3.4). | Couvert |
| Force brute et *credential stuffing* | 5 essais par code, 20 vérifications par quart d'heure et par IP, message d'échec générique, filtrage des adresses malveillantes par le WAF ; la route de connexion n'a pas encore de limiteur dédié. | Partiel |
| Stockage non sécurisé | Empreinte bcrypt salée (coût 12), jamais de mot de passe en clair ni de MD5. | Couvert |
| Gestion des sessions | Jeton JWT signé, imprévisible et nouveau à chaque connexion, conservé en mémoire et non dans l'URL ; compte re-vérifié à chaque requête. Pas encore de révocation individuelle ni de déconnexion globale des jetons. | Partiel |

## 3.4 Réalisation

### 3.4.1 Backend et frontend

Le backend est organisé en couches (listing IV.2, annexe IV) ; chaque route métier est protégée par la chaîne de *middlewares* de la figure 3.2 : `authenticate`, `authorize`, gardes de ressources (`checkClientAccess`, listing IV.3, `checkTaskAccess`, `checkDocumentAccess`), puis validation Zod. Parmi les mécanismes notables :

— la liste des clients est triée par ordre alphabétique ou par date directement par la base, et la fiche client consolide historique, notes, tâches liées et documents ;

— l'affectation d'une tâche déclenche un courriel et une notification interne à la personne assignée ;

— chaque facture est numérotée automatiquement et son circuit de signature électronique est historisé ;

— dans la DataRoom, les fichiers sont téléversés vers S3 après contrôle des droits, consultés par URL présignées (liens signés expirant au bout de quelques minutes), prévisualisés (PDF, Word, Excel, PowerPoint), et leur texte est extrait pour l'assistant IA.

Côté frontend, l'application React est structurée en pages et composants réutilisables ; le composant `PrivateRoute` masque les écrans non autorisés, simple commodité d'ergonomie, la sécurité effective étant assurée par l'API.

### 3.4.2 Présentation de l'application

Une barre latérale donne accès aux modules autorisés pour le profil connecté. Le tableau de bord (figure 3.4) réunit des actions rapides (nouveau client, tâche, facture, rendez-vous, document, question à l'assistant), les indicateurs clés et une vue d'ensemble des clients par segment.

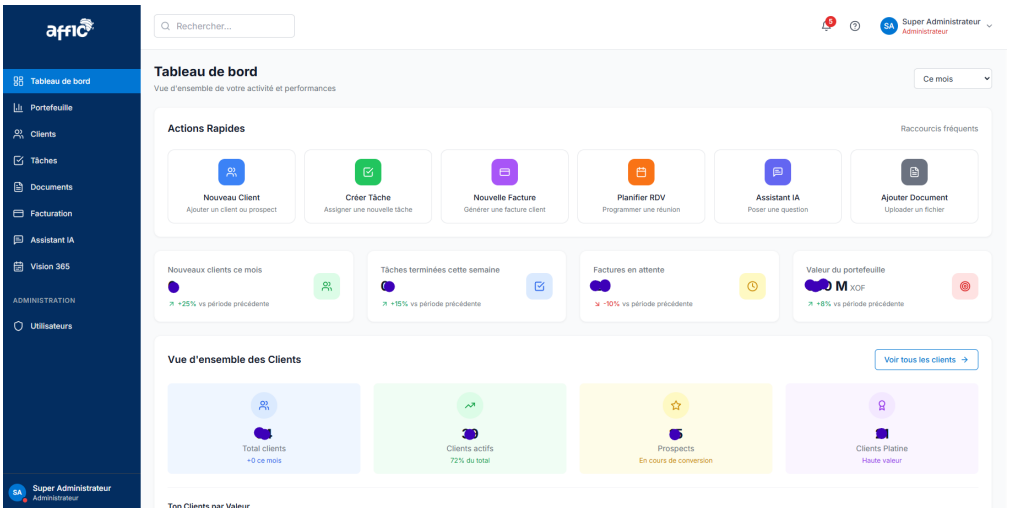



FIGURE 3.4 – Tableau de bord de la plateforme AFFIC

La gestion des clients (figure 3.5) présente la liste triable des clients avec leur score, leur responsable et leur dernier contact, et un formulaire de création. La gestion des tâches (figure 3.6) propose des indicateurs, des filtres par statut, priorité et personne assignée, et deux vues, liste et kanban.

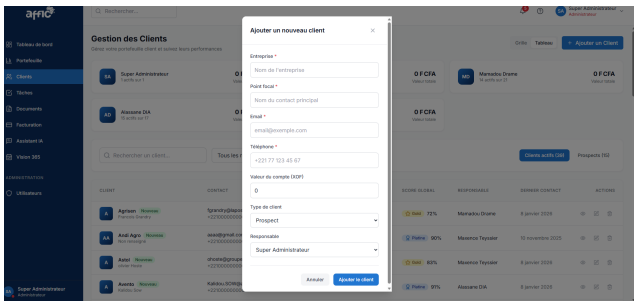



FIGURE 3.5 – Gestion des clients

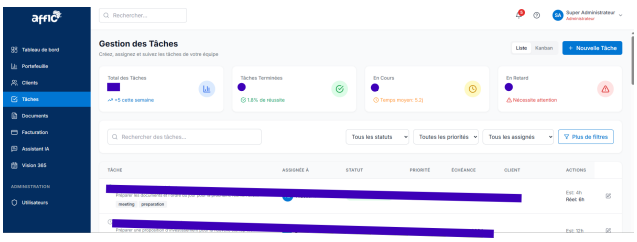



FIGURE 3.6 – Gestion des tâches

La DataRoom (figure 3.7) organise les documents en dossiers, avec recherche, tri, import, partage et renommage. L'assistant IA (figure 3.8) répond en langage naturel et propose des questions adaptées au profil. Les écrans du portefeuille et de la facturation figurent en annexe III ; les données des clients ont été masquées.

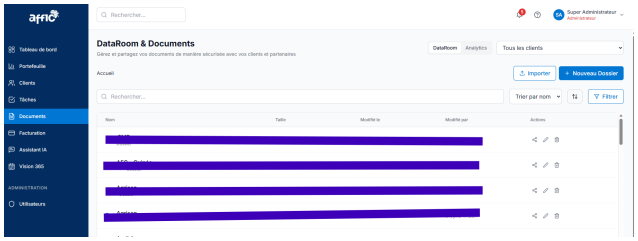



FIGURE 3.7 – DataRoom et gestion documentaire

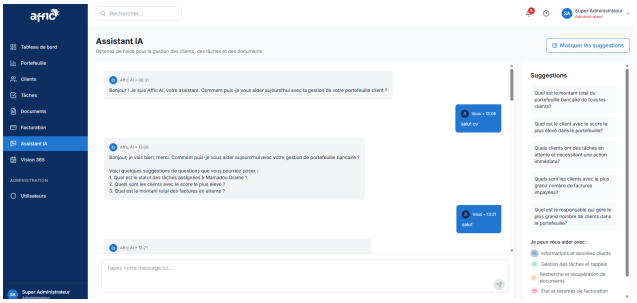



FIGURE 3.8 – Assistant IA conversationnel

### 3.4.3 Scoring client

Chaque client porte trois scores sur 100 : revenu ($S_R$), activité ($S_A$) et potentiel ($S_P$). Le score global affiché est la moyenne pondérée

$$S_G = 0{,}5\,S_R + 0{,}3\,S_A + 0{,}2\,S_P,$$

et la segmentation du tableau de bord classe le client en « Platine » ($S_R \geq 85$), « Gold » ($70 \leq S_R < 85$) ou « Silver » ($S_R < 70$). **Dans la version actuelle, les trois scores de base sont fictifs** : ils sont tirés au hasard entre 70 et 99 (`Math.random()`) à la création du client et ne sont jamais recalculés. Seules la formule d'agrégation et la segmentation sont donc opérationnelles ; le calcul de $S_R$, $S_A$ et $S_P$ à partir des données réelles (honoraires facturés, fréquence des interactions, montant du mandat) reste à définir avec le cabinet.

### 3.4.4 Assistant IA

L'assistant s'appuie sur l'API d'OpenAI : le modèle GPT-4 rédige les réponses, GPT-3.5-turbo génère les suggestions de questions, et le modèle `text-embedding-ada-002` produit les vecteurs de la recherche sémantique, stockés dans la base vectorielle Pinecone. Le service `enhancedChatService` oriente chaque question, selon ses mots-clés, vers les données structurées, les documents, ou les deux (approche RAG [14]).

Les données transmises au fournisseur sont les suivantes : la question et les trois derniers échanges de l'utilisateur ; pour les données structurées, un extrait d'au plus 20 clients (société, nom du contact, valeur du compte, score, statut, responsable), 20 tâches, 20 factures et 10 portefeuilles ; pour les documents, le texte extrait des fichiers lors de leur indexation, puis les cinq passages les plus pertinents au moment de la question. Aucun mot de passe, jeton ni fichier brut n'est envoyé.

Les habilitations sont appliquées *avant* l'appel au modèle, conformément aux recommandations de l'OWASP [5] : le backend construit lui-même le contexte avec des requêtes filtrées, de sorte que le modèle ne reçoit jamais une donnée que l'utilisateur ne pourrait pas consulter. Pour un collaborateur, seuls ses clients, ses tâches et les factures et portefeuilles de ses clients sont extraits, et la recherche Pinecone est restreinte par un filtre sur les identifiants de ses clients. Chaque échange est journalisé

(table `ChatLog`). Deux réserves subsistent : le profil lecture seule n'est pas encore filtré dans ce service, et l'envoi de données clients à un prestataire situé hors du Sénégal devra être encadré au regard de la loi n° 2008-12 [18].

## 3.5 Mesures de sécurité mises en œuvre

Le tableau 3.6 met les contre-mesures en regard des risques OWASP [3].

TABLEAU 3.6 – Contre-mesures implémentées par risque OWASP Top 10:2025

| **Risque OWASP 2025** | **Contre-mesures dans la plateforme** |
|---|---|
| A01 Broken Access Control | RBAC à quatre rôles ; gardes de ressources côté serveur ; refus par défaut. |
| A02 Security Misconfiguration | En-têtes Helmet ; CORS restrictif ; erreurs génériques ; bucket S3 privé ; WAF. |
| A03 Software Supply Chain Failures | Dépendances verrouillées (`package-lock.json`) ; image Docker officielle minimale ; analyse automatique des dépendances à intégrer. |
| A04 Cryptographic Failures | TLS de bout en bout ; bcrypt salé ; documents chiffrés au repos ; secrets hors du code. |
| A05 Injection | Requêtes paramétrées (Prisma) ; validation Zod ; encodage des sorties par React. |
| A06 Insecure Design | Critères de sécurité dans chaque récit utilisateur ; défense en profondeur. |
| A07 Authentication Failures | Politique de mots de passe ; double authentification ; jetons à durée limitée ; réponses non discriminantes. |
| A08 Software or Data Integrity Failures | Migrations versionnées ; signature électronique historisée des factures. |
| A09 Security Logging and Alerting Failures | Journaux d'activité et de sécurité horodatés et imputables ; alertes en temps réel à mettre en place. |
| A10 Mishandling of Exceptional Conditions | Gestionnaire d'erreurs centralisé ; messages d'erreur génériques ; délais d'attente sur les appels externes. |

## 3.6 Tests

### 3.6.1 Tests de sécurité

La qualité a été vérifiée à trois niveaux [15] : analyse statique (TypeScript, ESLint, SonarQube), tests de sécurité des API et recette utilisateur en fin de sprint. Les tests de sécurité ciblent les risques OWASP A01, A05 et A07 (édition 2025). Ils sont automatisés dans un script Node.js (`run-security-tests.m` listing IV.4) qui génère un jeton de courte durée pour chaque rôle, envoie des requêtes construites pour contourner une protection, compare le code HTTP obtenu au code attendu et enregistre requêtes et réponses comme preuves ; il peut être rejoué après chaque modification (non-régression). Pour ne jamais attaquer la production, les tests ont été exécutés le 12 octobre 2026 sur une base de test, restaurée à partir d'une sauvegarde de la base de la plateforme (54 clients) ; le tableau 3.7 présente les résultats.

TABLEAU 3.7 – Cas de tests de sécurité et résultats obtenus

| Cas de test | OWASP | Identité | Attendu | Obtenu | Verdict |
|---|---|---|---|---|---|
| Accès à l'API sans jeton | A07 | Anonyme | 401 | 401 | Conforme |
| Jeton altéré (signature invalide) | A07 | Admin (jeton modifié) | 401 | 401 | Conforme |
| Accès au client d'un autre collaborateur | A01 | Collaborateur | 403 | 403 | Conforme |
| Corps de requête malformé | A05 | Anonyme | 400 | 400 | Conforme |
| Injection SQL `' OR '1'='1' --` dans la recherche | A05 | Admin | Aucun effet | 200, 0 résultat sur 54 | Conforme |
| Trois échecs de connexion successifs | A07 | Anonyme | 401 à chaque essai | 401, 401, 401 | Conforme |
| Création par un profil lecture seule | A01 | Lecture seule | 403 | — | Non exécuté |
| Accès direct à l'URL S3 d'un document | A01 | Anonyme | Accès refusé | — | Non exécuté |

Les tests mobilisent trois identités, et non le seul administrateur : un visiteur non authentifié, un collaborateur et un administrateur. Le collaborateur sert à vérifier le cloisonnement entre portefeuilles ; l'administrateur, qui voit les 54 clients, représente le cas le plus défavorable pour l'injection. Les six cas exécutés sont conformes (captures en annexe V, figures V.1 à V.6). Le refus du collaborateur, pourtant authentifié et doté du bon rôle, valide le contrôle au niveau de la ressource contre la manipulation d'identifiant (IDOR) ; la chaîne d'injection, transmise par Prisma comme simple paramètre, est recherchée telle quelle.

Deux outils spécialisés ont complété ce script. `sqlmap` [26], lancé avec un jeton valide sur la recherche de clients (listing IV.5), a essayé des centaines de charges d'injection (booléenne, temporelle,

par erreur, `UNION`, requêtes empilées) et conclut qu'aucun paramètre n'est injectable. Avec le proxy d'interception Burp Suite [27], des requêtes réelles ont été capturées puis rejouées après modification (figures V.7 et V.8) : la charge d'injection reste sans effet, et les requêtes rejouées sur la ressource d'un autre portefeuille ou sur une route d'administration avec un jeton non habilité reçoivent un code 403.

### 3.6.2 Analyse de la qualité du code avec SonarQube

SonarQube [25], plateforme d'analyse statique présentée en annexe VI, a été déployé localement sous Docker et appliqué au frontend et au backend, soit environ 7 600 lignes (listing IV.6). La première analyse, du 11 octobre 2026 (figure VI.1, annexe VI), a été validée par la *Quality Gate* mais a relevé cinq bogues (note C) et dix points sensibles de sécurité non examinés (note E). Les bogues ont été corrigés (tableau 3.8) et chaque point sensible a reçu un statut justifié (tableau 3.9).

TABLEAU 3.8 – Bogues détectés par SonarQube et corrections

| **Fichier** | **Problème** | **Correction** |
|---|---|---|
| `emailService.ts` | Opérateur conditionnel renvoyant la même valeur dans les deux branches | Condition et variable inutilisée supprimées |
| `documentExtraction-Service.ts` (2), `vectorService.ts` (2) | Caractères de contrôle écrits en dur dans une expression régulière | Remplacement par la classe Unicode `\p{Cc}` |

TABLEAU 3.9 – Revue des points sensibles de sécurité

| **Emplacement** | **Point signalé** | **Décision et justification** |
|---|---|---|
| `routes/clients.ts` | `Math.random()` | Sûr sur le plan cryptographique : sert seulement à générer les scores *fictifs* des clients (section 3.4.3), jamais un secret ni un jeton. |
| `documentExtraction-Service.ts`, `vectorService.ts` | Expressions régulières (ReDoS) | Sûr : classes `[^>]` et `[^<]` disjointes, sans retour arrière exponentiel. |
| `routes/documents.ts` | Téléversement | Sûr : 50 Mo maximum (`multer`), bucket S3 privé. |
| `routes/users.ts` | Mots de passe | Sûr : bcrypt, 12 tours, sel. |
| `server.ts` | Configuration CORS | Sûr : liste blanche d'origines, `helmet`. |
| `server.ts` | URL `http://` du frontend | Risque reconnu : HTTPS via CloudFront. |
| `scripts/seed.ts` | Mots de passe en dur | Risque reconnu : comptes de démonstration, à externaliser. |

La seconde analyse, du 12 octobre 2026 (figure VI.2, annexe VI), porte sur le code modifié depuis la première (onglet *New Code* de SonarQube) ; le tableau 3.10 compare les deux.

TABLEAU 3.10 – Résultats SonarQube avant et après correction

| **Indicateur** | **1re analyse (11/10)** | **2de analyse (12/10)** |
|---|---|---|
| Bogues (fiabilité) | 5 (C) | 0 (A) |
| Vulnérabilités (sécurité) | 0 (A) | 0 (A) |
| Points sensibles (revue) | 10, 0 % revus (E) | 0 nouveau (A) |
| Mauvaises pratiques | 106, dette 5 h 1 min (A) | 0 nouvelle, dette 0 (A) |
| Duplication | 0,9 % | 0,0 % |
| Couverture de tests | 0 % | 0 % |
| Quality Gate | Passed | Passed |

Les corrections n'ont introduit ni bogue, ni vulnérabilité, ni mauvaise pratique ; la couverture reste nulle, faute de tests unitaires automatisés.

## 3.7 Déploiement en production

La plateforme est déployée sur des services managés AWS en région `eu-west-1` (topologie en annexe VII, figure VII.1). Le frontend est servi depuis S3 et distribué par CloudFront, qui impose HTTPS ; le backend, empaqueté en image Docker exécutée sous un utilisateur non privilégié (listing IV.7), tourne sur ECS Fargate derrière un répartiteur de charge ; la base PostgreSQL est hébergée sur RDS et les documents dans un bucket S3 privé. Les secrets sont stockés chiffrés dans AWS Systems Manager Parameter Store, deux rôles IAM distincts appliquent le moindre privilège, et les conteneurs ne sont accessibles que depuis le répartiteur. Après le déploiement, un pare-feu applicatif web (AWS WAF) a été associé à la distribution CloudFront (figure VII.2, annexe VII). Il applique la règle gérée par AWS de réputation des adresses IP, qui écarte les requêtes provenant d'adresses connues pour des activités malveillantes, et son tableau de bord comptabilise le trafic autorisé et bloqué. À partir de deux semaines de trafic, AWS recommande en outre d'activer des règles contre les requêtes visant des vulnérabilités applicatives, les robots, les adresses anonymes et les sources de déni de service ; leur activation est prévue. Le déploiement est automatisé (listing IV.8) et supervisé par CloudWatch. Le tableau 3.11 donne l'ordre de grandeur du coût mensuel.

TABLEAU 3.11 – Estimation des coûts mensuels de l'infrastructure AWS (1 € = 655,957 FCFA)

| **Service** | **Configuration** | **Coût/mois** |
|---|---|---|
| ECS Fargate | 0,5 vCPU, 1 Go RAM | ~9 800 FCFA |
| RDS PostgreSQL | Instance db.t3.micro | ~13 100 FCFA |
| S3 + CloudFront | 50 Go de stockage et transfert | ~6 600 FCFA |
| ALB | Répartiteur de charge | ~13 100 FCFA |
| **Total** | | **~42 600 FCFA** |

## 3.8 Points non encore validés

Les éléments suivants ne sont pas encore validés et constituent des limites du travail présenté :

— **Deux tests de sécurité non exécutés** (tableau 3.7) : la tentative de création par un profil lecture seule, faute de compte de ce rôle dans la base, et l'accès direct à l'URL S3 d'un document, faute de document de test. Ils seront exécutés après création de ces données.

— **Assistant IA non testé** : aucun test n'a vérifié l'étanchéité des habilitations dans ses réponses, ni sa résistance aux injections de requête (*prompt injection*) ; de plus, le profil lecture seule n'y est pas encore filtré (section 3.4.4).

— **Limiteur de débit trop permissif** : la limite globale de 5 000 requêtes par fenêtre de 45 minutes ne freine pas une attaque par force brute ; un limiteur dédié à la route de connexion (par exemple 5 tentatives par quart d'heure) est prioritaire.

— **Comptes de démonstration sans double authentification** : les cinq comptes de recette en sont dispensés et leurs mots de passe figurent dans le script d'initialisation ; ils devront être supprimés ou soumis au second facteur avant une utilisation réelle.

— **État de la double authentification en mémoire** : le nombre d'essais restants et l'usage unique de chaque code sont suivis dans la mémoire du serveur. Avec plusieurs conteneurs derrière le répartiteur, ou après un redémarrage, ces limites pourraient être contournées ; cet état devra être déplacé dans un stockage partagé (base de données ou cache Redis).

— **Mesures et scores** : disponibilité et temps de réponse restent des objectifs non mesurés, et les scores clients sont fictifs (section 3.4.3).

# Conclusion

Ce chapitre a traduit les besoins en une solution déployée sur AWS : choix justifiés, RBAC renforcé au niveau des ressources, double authentification et mesures vérifiées par des tests, sous réserve des points de la section 3.8.

# Conclusion générale et perspectives

## Bilan du travail réalisé

Ce mémoire cherchait à doter le cabinet AFFIC d'une plateforme unique de gestion de portefeuille client, tout en garantissant un niveau de sécurité adapté à la confidentialité du secteur financier.

Pour y répondre, nous avons combiné la méthode agile Scrum, les formalismes UML et Merise, et le principe de sécurité dès la conception, adossé à l'OWASP Top 10. La plateforme livrée réunit désormais clients, mandats, tâches, facturation, documents et pilotage dans un même outil, complété par un assistant IA dont le contexte est filtré selon le profil de l'utilisateur ; ce filtrage reste toutefois à étendre au profil lecture seule et à valider par des tests.

Sur le plan de la sécurité, la réponse repose sur une défense en profondeur : chaque connexion exige un mot de passe puis un code à usage unique reçu par courriel ; chaque requête est authentifiée, soumise à un contrôle de rôle puis de ressource, validée et journalisée ; les mots de passe sont hachés et salés ; les documents ne sont accessibles que par des liens temporaires. Les tests menés sur la grille OWASP, complétés par sqlmap et Burp Suite, ont confirmé l'efficacité des mesures testées. Certains points restent toutefois à valider : deux tests non exécutés, l'assistant IA non encore testé, un limiteur de débit trop permissif, des comptes de démonstration sans double authentification et des scores clients encore fictifs.

## Apport personnel du projet

Ce projet m'a d'abord permis de mobiliser, sur un cas réel, l'ensemble des compétences acquises durant le Master : ingénierie des besoins, modélisation, architecture logicielle, développement full-stack TypeScript, services cloud et, surtout, sécurité applicative dans un contexte où les données sont hautement sensibles.

Il m'a ensuite appris à justifier des choix techniques face à des alternatives plutôt que de les adopter par habitude, et à déployer et exploiter une application en production sur AWS. Sur le plan humain, la conduite du projet en Scrum, au contact des utilisateurs du cabinet, m'a appris à reformuler des besoins métier, à prioriser sous contrainte de temps et à rendre compte de l'avancement. Enfin, ce travail m'a convaincu que la sécurité ne se décrète pas en fin de projet : elle se conçoit, se construit et se vérifie à chaque étape.

# Perspectives

Plusieurs axes d'évolution se dégagent. Le premier concerne le renforcement de l'authentification : évolution de la double authentification vers une application TOTP ou des clés de sécurité matérielles, sa généralisation aux comptes de démonstration, longueur minimale des mots de passe portée à douze caractères avec vérification contre des listes de mots de passe compromis, limitation spécifique des tentatives de connexion et détection des connexions inhabituelles. Le deuxième porte sur la sécurité opérationnelle : intégration de SonarQube et d'analyses de vulnérabilités des dépendances dans une chaîne d'intégration continue, tests d'intrusion périodiques, activation des règles WAF recommandées par AWS et alertes en temps réel. Le troisième concerne le scoring, dont les scores devront être calculés à partir des données réelles du cabinet. Le quatrième vise la performance, avec un cache et des files de traitement asynchrone pour les courriels et l'indexation. Le cinquième relève de l'extension fonctionnelle : application mobile, intégrations comptables et bancaires, analytique avancée. Enfin, l'assistant IA devra être testé (étanchéité des habilitations, injections de requête), étendu au filtrage du profil lecture seule, et l'envoi de données à son fournisseur encadré, par exemple par pseudonymisation.

# RÉFÉRENCES BIBLIOGRAPHIQUES


[1] R. Reix, B. Fallery, M. Kalika et F. Rowe, *Systèmes d'information et management des organisations*, 7e édition, Vuibert, 2016.

[2] E. Peelen, F. Jallat, E. Stevens et P. Volle, *Gestion de la relation client*, 4e édition, Pearson, 2014.

[3] OWASP Foundation, *OWASP Top 10:2025 – The Ten Most Critical Web Application Security Risks*, 2025. Disponible sur : `https://top10.owasp.org/2025/` (consulté le 17 octobre 2026).

[4] OWASP Foundation, *A07:2025 – Authentication Failures*, OWASP Top 10:2025. Disponible sur : `https://top10.owasp.org/2025/A07_2025-Authentication_Failures/` (consulté le 17 octobre 2026).

[5] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*, 2023. Disponible sur : `https://owasp.org/www-project-top-10-for-large-language-model-applications/` (consulté le 5 octobre 2026).

[6] ANSSI, *Guide d'hygiène informatique : renforcer la sécurité de son système d'information en 42 mesures*, Agence nationale de la sécurité des systèmes d'information, 2017.

[7] P. A. Grassi *et al.*, *Digital Identity Guidelines: Authentication and Lifecycle Management*, NIST Special Publication 800-63B, National Institute of Standards and Technology, 2017. Disponible sur : `https://pages.nist.gov/800-63-3/sp800-63b.html` (consulté le 5 octobre 2026).

[8] M. Jones, J. Bradley et N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, mai 2015.

[9] N. Provos et D. Mazières, « A Future-Adaptable Password Scheme », *Proceedings of the USENIX Annual Technical Conference*, 1999.

[10] R. Sandhu et P. Samarati, « Access Control: Principle and Practice », *IEEE Communications Magazine*, vol. 32, no 9, 1994, p. 40–48.

[11] R. Sandhu, E. Coyne, H. Feinstein et C. Youman, « Role-Based Access Control Models », *IEEE Computer*, vol. 29, no 2, 1996, p. 38–47.

[12] V. C. Hu *et al.*, *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, NIST Special Publication 800-162, National Institute of Standards and Technology, 2014.

[13] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, thèse de doctorat, University of California, Irvine, 2000.

[14] P. Lewis *et al.*, « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks », *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[15] I. Sommerville, *Software Engineering*, 10e édition, Pearson, 2016.

[16] K. Schwaber et J. Sutherland, *The Scrum Guide*, 2020. Disponible sur : `https://scrumguides.org` (consulté le 5 octobre 2026).

[17] M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

[18] République du Sénégal, *Loi n° 2008-12 du 25 janvier 2008 portant sur la protection des données à caractère personnel*, Journal officiel du Sénégal, 2008.

[19] Meta Open Source, *React – Documentation officielle*. Disponible sur : `https://react.dev` (consulté le 5 octobre 2026).

[20] Prisma Data, *Prisma ORM Documentation*. Disponible sur : `https://www.prisma.io/docs` (consulté le 5 octobre 2026).

[21] OpenJS Foundation, *Express – Node.js web application framework*. Disponible sur : `https://expressjs.com` (consulté le 5 octobre 2026).

[22] PostgreSQL Global Development Group, *PostgreSQL Documentation*. Disponible sur : `https://www.postgresql.org/docs/` (consulté le 5 octobre 2026).

[23] Amazon Web Services, *AWS Documentation : Amazon S3, Amazon ECS, Amazon CloudFront*. Disponible sur : `https://docs.aws.amazon.com` (consulté le 5 octobre 2026).

[24] C. McDonnell, *Zod – TypeScript-first schema validation*. Disponible sur : `https://zod.dev` (consulté le 5 octobre 2026).

[25] SonarSource, *SonarQube Documentation*. Disponible sur : `https://docs.sonarsource.com/sonarqube/` (consulté le 5 octobre 2026).

[26] sqlmap, *Automatic SQL injection and database takeover tool*. Disponible sur : `https://sqlmap.org` (consulté le 12 octobre 2026).

[27] PortSwigger, *Burp Suite*. Disponible sur : `https://portswigger.net/burp` (consulté le 12 octobre 2026).

[28] Salesforce, *What is Salesforce?* Disponible sur : `https://www.salesforce.com/fr/products/what-is-salesforce/` (consulté le 15 octobre 2026).

[29] Microsoft, *Microsoft Dynamics 365*. Disponible sur : `https://www.microsoft.com/dynamics-365` (consulté le 15 octobre 2026).

[30] HubSpot, *HubSpot CRM*. Disponible sur : `https://www.hubspot.com/products/crm` (consulté le 15 octobre 2026).

[31] Odoo, *Odoo, logiciels de gestion open source*. Disponible sur : `https://www.odoo.com` (consulté le 15 octobre 2026).

[32] Intralinks, *Virtual Data Rooms*. Disponible sur : `https://www.intralinks.com` (consulté le 15 octobre 2026).

[33] Drooms, *Secure Data Room*. Disponible sur : `https://www.drooms.com` (consulté le 15 octobre 2026).

# Annexe I

# Extrait du schéma de données (Prisma)

Le listing I.1 présente un extrait du schéma de données décrit avec Prisma, qui constitue la source unique de vérité du modèle physique.

Listing I.1 – Extrait du fichier schema.prisma

```
model User {
  id         String   @id @default(cuid())
  email      String   @unique
  password   String            // hash bcrypt
  role       UserRole @default(READONLY)
  isActive   Boolean  @default(true)
  lastLogin  DateTime?
  // relations : clients, taches, factures, fichiers, sessions...
}
enum UserRole { ADMIN  MANAGER  COLLABORATOR  READONLY }
model Client {
  id              String       @id @default(cuid())
  name            String
  company         String
  email           String       @unique
  status          ClientStatus @default(PENDING)
  clientType      ClientType   @default(PROSPECT)
  accountValue    BigInt       @default(0)
  revenueScore    Int          @default(0)
  activityScore   Int          @default(0)
  potentialScore  Int          @default(0)
  responsibleId   String       // FK -> User (responsable)
  portfolio       Portfolio?
  tasks           Task[]
  invoices        Invoice[]
  files           File[]
  notes           ClientNote[]
} // idem : File, FileAccess, Portfolio, Task, Invoice, Event...
```

# Annexe II

# Principaux points de terminaison de l'API

Le tableau II.1 recense les principaux points de terminaison exposés par l'API REST.

TABLEAU II.1 – Points de terminaison de l'API REST (extrait)

| Verbe | Route | Description |
|---|---|---|
| POST | /api/auth/login | Connexion, délivrance du JWT |
| GET | /api/auth/me | Profil de l'utilisateur courant |
| GET | /api/clients | Liste des clients (filtres, tri) |
| POST | /api/clients | Création d'un client |
| GET | /api/clients/:id/history | Historique d'un client |
| GET | /api/tasks | Liste des tâches |
| POST | /api/tasks | Création et affectation d'une tâche |
| GET | /api/tasks/metrics/summary | Métriques des tâches |
| GET | /api/invoices | Liste des factures |
| PUT | /api/invoices/:id/pay | Marquer une facture payée |
| GET | /api/documents/folders | Arborescence de la DataRoom |
| POST | /api/documents/upload | Téléversement vers S3 |
| POST | /api/documents/files/:id/share | Partage nominatif d'un fichier |
| GET | /api/portfolio | Vue d'ensemble des portefeuilles |
| POST | /api/chat | Question à l'assistant IA |
| GET | /api/chat/history | Historique des conversations |
| GET | /health | Contrôle de disponibilité |

# Annexe III

# Captures d'écran de la plateforme

Les figures ci-après complètent la présentation de l'application faite au chapitre 3. Par souci de confidentialité, les données réelles des clients (noms, contacts, montants) visibles sur ces captures ont été masquées.

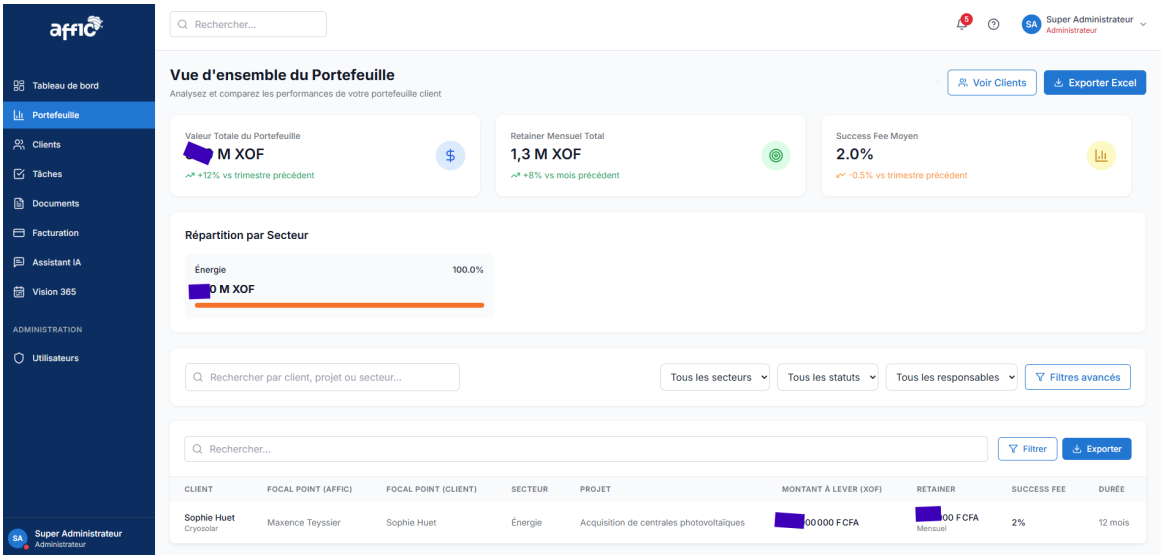



FIGURE III.1 – Vue d'ensemble du portefeuille

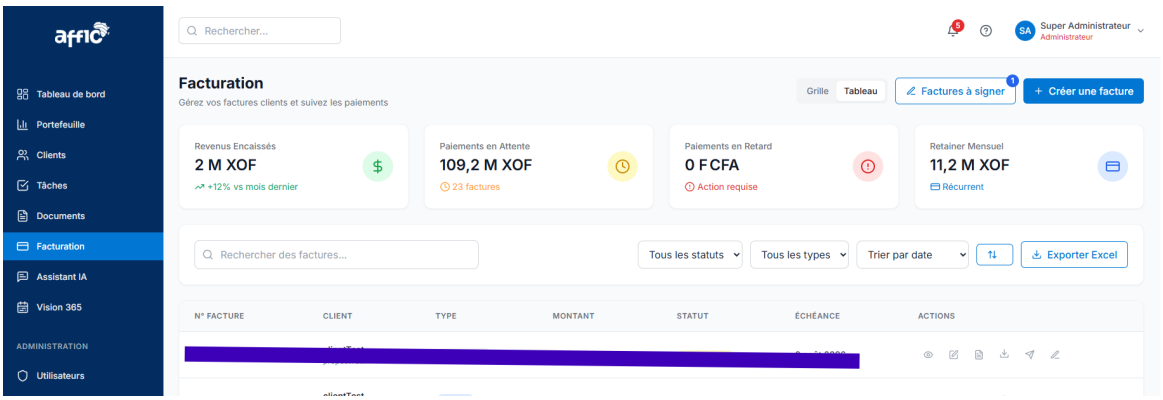



FIGURE III.2 – Module de facturation

# Annexe IV

# Extraits de code source

Cette annexe regroupe les extraits de code référencés au chapitre 3, déplacés ici pour alléger le corps du texte.

Listing IV.1 – Génération du code et du jeton de défi (`twoFactorService.ts`, extrait)

```
const code = crypto.randomInt(0, 1_000_000).toString().padStart(6, '0');
const jti  = crypto.randomUUID();
const challengeToken = jwt.sign(
  { purpose: 'login-2fa', uid: userId, h: hashCode(jti, code) },
  secret(),
  { expiresIn: 10 * 60, jwtid: jti }      // valable 10 minutes
);
```

Listing IV.2 – Arborescence du backend

```
backend/src/
  middleware/      # authentification JWT, controle d'acces, validation,
     erreurs
  routes/          # auth, clients, tasks, invoices, documents,
                   # portfolio, events, users, chat, notifications
  services/        # s3Service, emailService, chatService,
                   # vectorService, activityService, notificationService
  scripts/         # seed et utilitaires
  server.ts        # point d'entree, montage des middlewares
```

Listing IV.3 – Middleware de contrôle d'accès aux clients (extrait)

```
export const checkClientAccess = async (req, res, next) => {
  const { clientId } = req.params;
  const user = req.user!;
  // Admin et Manager : acces a tous les clients
  if (user.role === 'ADMIN' || user.role === 'MANAGER') return next();
  // Collaborateur : uniquement ses clients assignes
  if (user.role === 'COLLABORATOR') {
    const client = await prisma.client.findFirst({
```

```
      where: { id: clientId, responsibleId: user.id },
    });
    if (!client) {
      return res.status(403).json({
        error: 'Une erreur est survenue.' });
    }
  }
  next();
};
```

Listing IV.4 – Extrait du script de tests : contrôle d'accès au niveau de la ressource

```
// Clients visibles par l'administrateur et par le collaborateur
const all  = list((await call('GET', '/clients', { token: admin })).data);
const mine = new Set(list((await call('GET', '/clients',
                    { token: collab })).data).map((c) => c.id));
// Un client qui n'est PAS dans le portefeuille du collaborateur
const other = all.find((c) => !mine.has(c.id));
// Tentative d'accès direct par son identifiant
r = await call('GET', `/clients/${other.id}`, { token: collab });
record('test_collaborateur_403', "Collaborateur accédant au client d'un
    autre",
        '403 Forbidden', r.status === 403, r);
```

Listing IV.5 – Lancement de sqlmap sur la route de recherche des clients

```
sqlmap -u "http://localhost:3001/api/clients?search=test*" \
  -H "Authorization: Bearer <jeton>" -H "Origin: http://localhost:5173" \
  --dbms=postgresql --batch --level=3 --risk=2
```

Listing IV.6 – Configuration de l'analyse SonarQube (`sonar-project.properties`)

```
sonar.projectKey=affic-app
sonar.sources=src,backend/src
sonar.exclusions=**/node_modules/**,**/dist/**,**/prisma/migrations/**
sonar.typescript.tsconfigPaths=tsconfig.app.json,backend/tsconfig.json
```

Listing IV.7 – Dockerfile du backend (extrait)

```
FROM node:18-alpine

# Utilisateur non privilegie (defense en profondeur)
RUN addgroup -g 1001 -S nodejs && adduser -S affic -u 1001
WORKDIR /app

# Dependances installees avant le code source (cache de build)
COPY --chown=affic:nodejs package*.json ./
RUN npm ci --only=production
COPY --chown=affic:nodejs . .
```

```
RUN npm run build
USER affic

EXPOSE 3001
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD curl -f http://localhost:3001/health || exit 1
CMD ["npm", "start"]
```

Listing IV.8 – Script de déploiement automatisé du backend (extrait)

```
set -e
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
ECR_URI="$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/affic-backend"

docker build -t affic-backend backend/
aws ecr get-login-password --region $REGION \
  | docker login --username AWS --password-stdin $ECR_URI
docker tag affic-backend:latest $ECR_URI:latest
docker push $ECR_URI:latest

aws ecs update-service --cluster affic-cluster \
  --service affic-backend-service --force-new-deployment
```

# Annexe V

# Captures des tests de sécurité

Cette annexe rassemble les captures d'écran des tests de sécurité commentés à la section 3.6.1. Seul le tableau de synthèse des résultats est conservé dans le corps du texte.




FIGURE V.1 – Test d'accès à l'API sans jeton : réponse 401

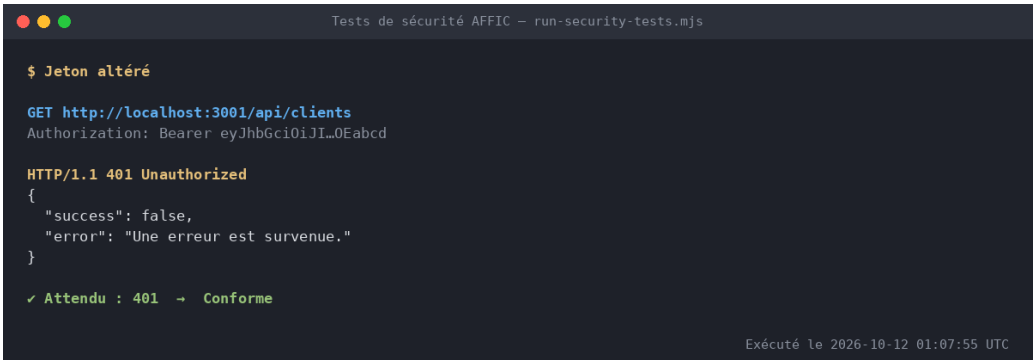



FIGURE V.2 – Test d'utilisation d'un jeton altéré : réponse 401




FIGURE V.3 – Test d'accès d'un collaborateur au client d'un autre : réponse 403




FIGURE V.4 – Test d'envoi d'une requête malformée : réponse 400




FIGURE V.5 – Test de tentative d'injection SQL : aucun effet




FIGURE V.6 – Test d'échecs de connexion répétés : réponse 401 à chaque tentative

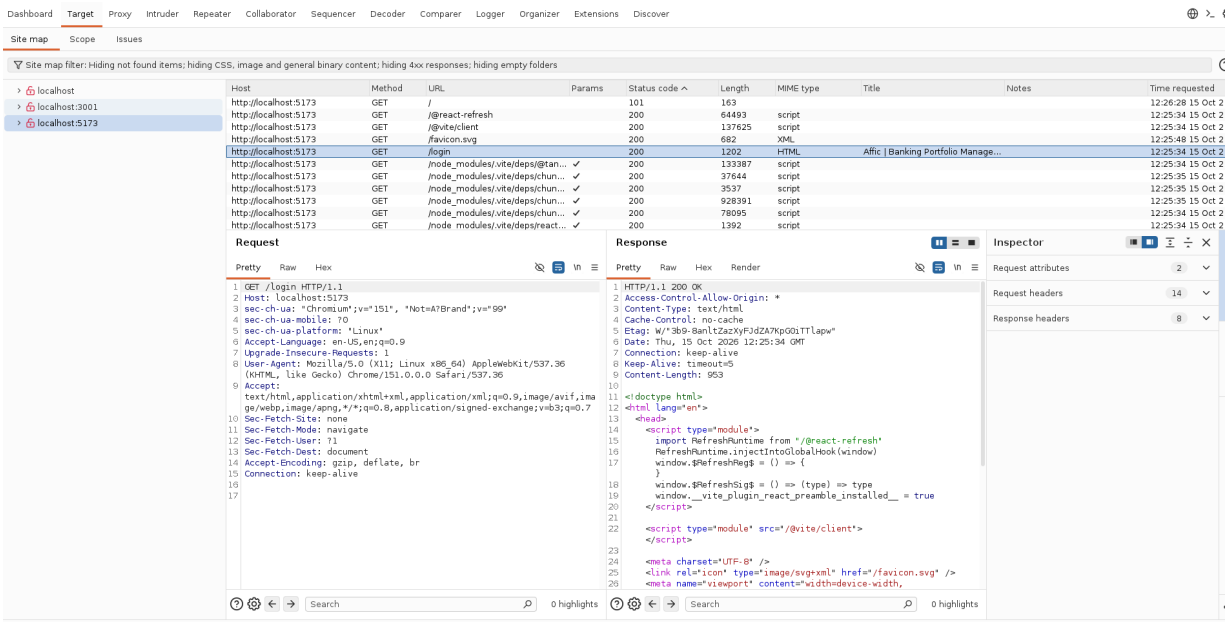



FIGURE V.7 – Interception du trafic de la plateforme dans Burp Suite

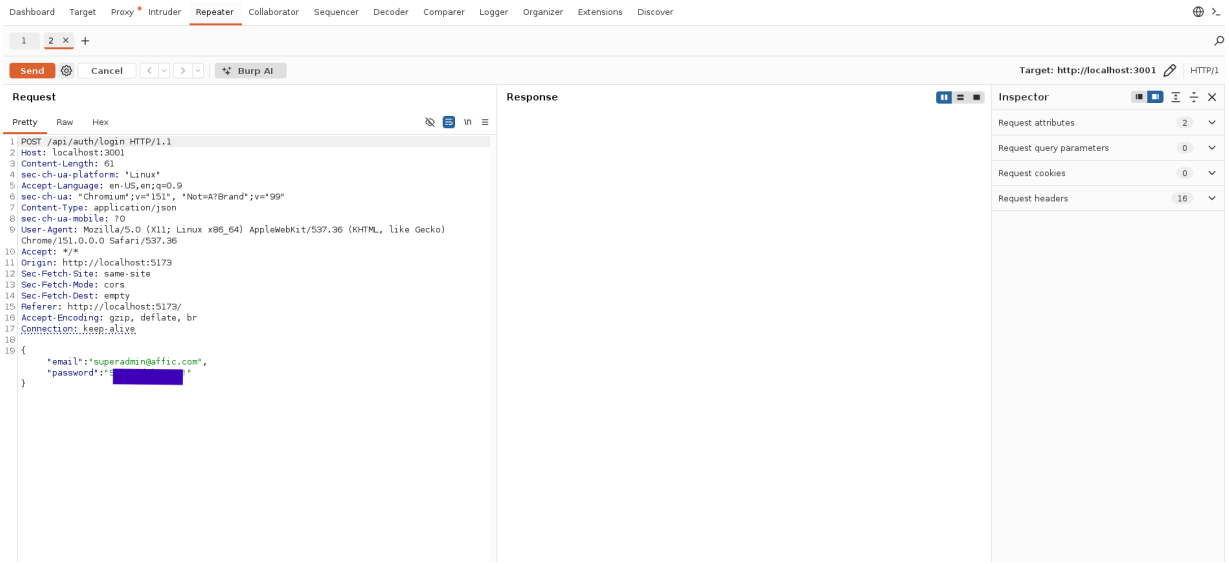



FIGURE V.8 – Modification d'une requête dans Burp Repeater : l'injection SQL reste sans effet

# Annexe VI

# Présentation de SonarQube

SonarQube [25] est une plateforme d'*analyse statique* du code source : il examine le code sans l'exécuter, en le confrontant à plusieurs centaines de règles propres à chaque langage (ici TypeScript et JavaScript). Il complète les tests dynamiques présentés plus haut, qui vérifient le comportement de l'application en fonctionnement, en détectant des défauts invisibles à l'exécution. Il classe ses constats en cinq familles :

— **Bogues** (fiabilité) : code dont le comportement est probablement erroné ;

— **Vulnérabilités** (sécurité) : failles exploitables avérées ;

— **Points sensibles de sécurité** (*security hotspots*) : code touchant à la sécurité (cryptographie, configuration CORS, téléversement de fichiers, expressions régulières…) qui doit être relu par un humain pour décider s'il présente un risque ;

— **Mauvaises pratiques** (*code smells*, maintenabilité) : code correct mais difficile à faire évoluer, dont le coût de correction est exprimé en *dette technique* ;

— **Duplication et couverture** : part du code copié-collé et part du code exercée par des tests automatisés.

Chaque dimension reçoit une note de A (meilleure) à E. Un ensemble de seuils, la *Quality Gate*, détermine si le projet est acceptable (*Passed*) ou non (*Failed*). Pour la note de revue de sécurité, la note dépend du pourcentage de points sensibles examinés : A à partir de 80 %, E en dessous de 30 %.

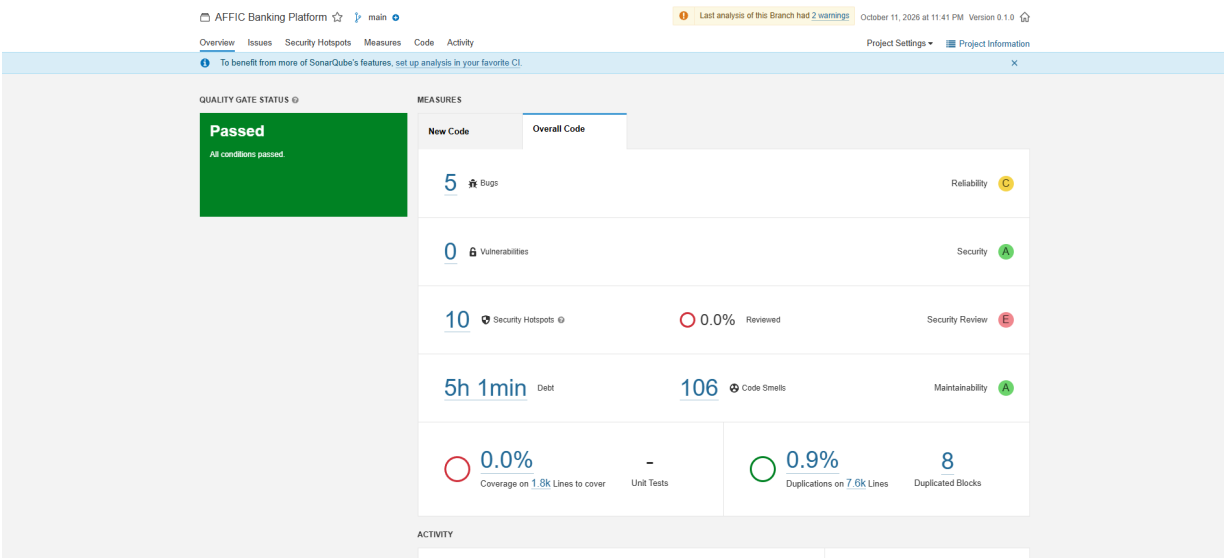



FIGURE VI.1 – Tableau de bord SonarQube lors de la première analyse (11 octobre 2026, code global)

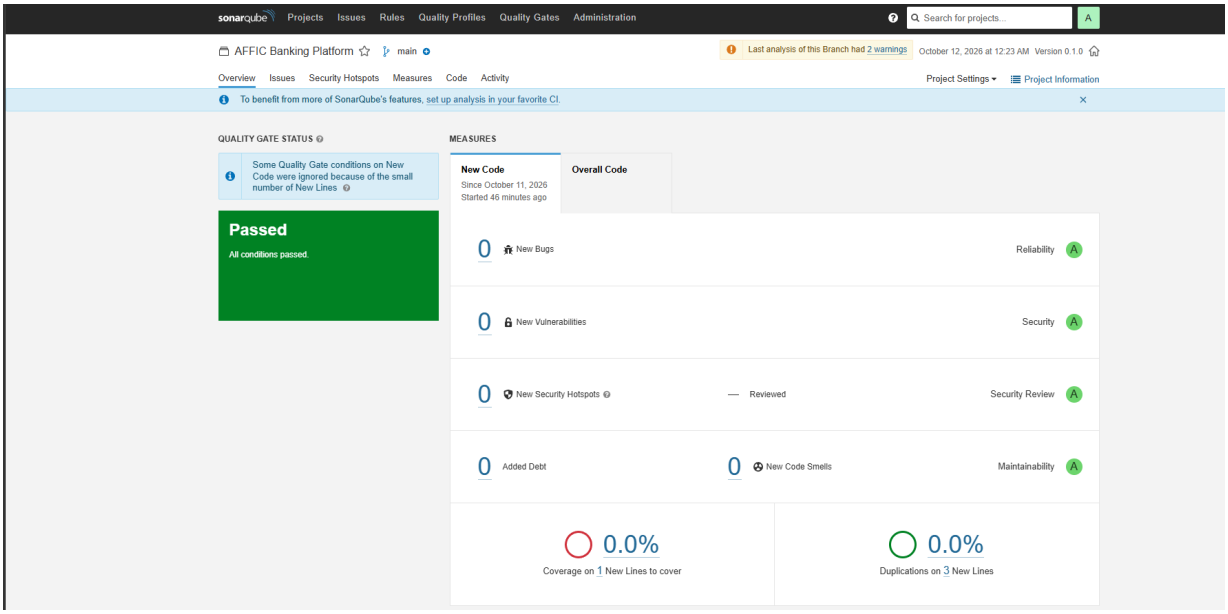



FIGURE VI.2 – Tableau de bord SonarQube lors de la seconde analyse (12 octobre 2026, code modifié)

# Annexe VII

# Déploiement sur AWS

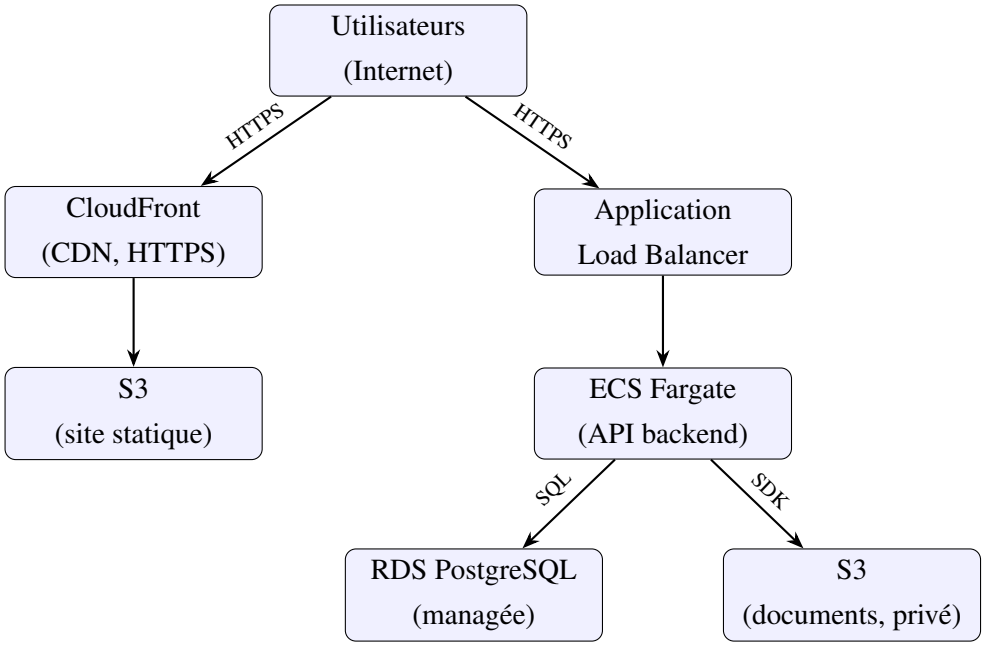



FIGURE VII.1 – Topologie de déploiement de la plateforme sur AWS

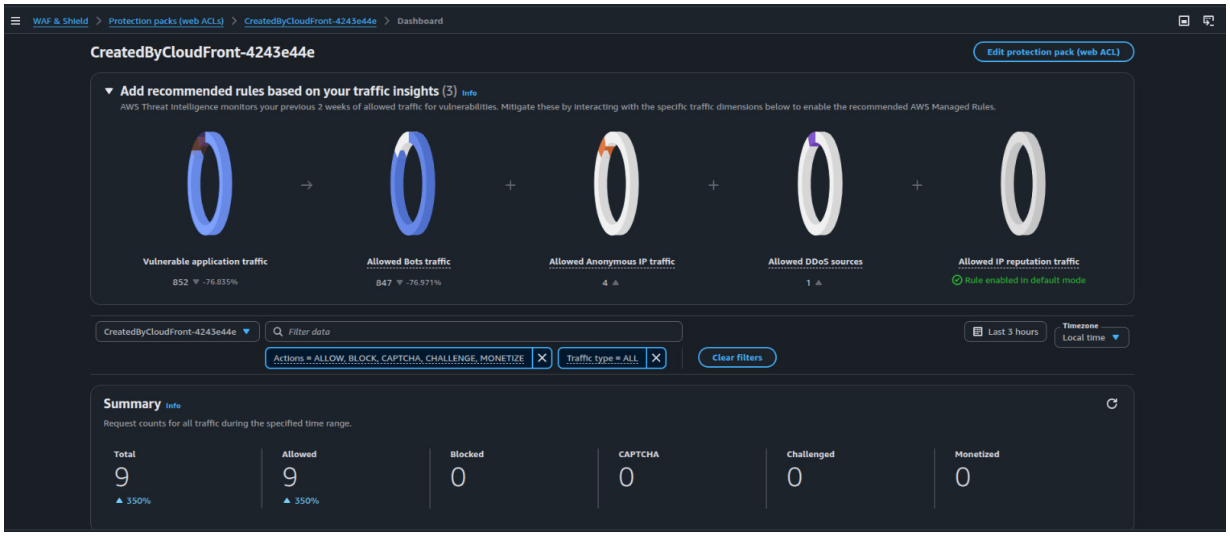



FIGURE VII.2 – Tableau de bord du pare-feu applicatif AWS WAF associé à la distribution CloudFront

## Résumé

La gestion du portefeuille client représente un enjeu stratégique pour les cabinets de conseil financier. Ce mémoire présente la conception et la réalisation d'AFFIC, une plateforme web sécurisée de gestion de portefeuille client intégrant un assistant conversationnel basé sur l'intelligence artificielle, développée au sein de la startup DataBeez. La solution centralise les informations clients, automatise le suivi des tâches et de la facturation, et sécurise l'accès aux documents sensibles grâce à un contrôle strict des droits et à la protection des données. Déployée en production sur AWS, la plateforme a vu sa sécurité vérifiée par des tests automatisés, sqlmap, Burp Suite et SonarQube ; quelques points, dont l'évaluation de l'assistant IA, restent à valider.



Mots-clés : gestion de portefeuille client, plateforme web sécurisée, assistant IA, protection des données.



## Abstract

Client portfolio management is a major strategic challenge for financial advisory firms. This thesis presents the design and implementation of AFFIC, a secure web platform for client portfolio management integrating an AI-powered conversational assistant, developed within the startup DataBeez. The solution centralizes client information, automates task and invoicing follow-up, and secures access to sensitive documents through strict access control and data protection. Deployed to production on AWS, the platform had its security verified through automated tests, sqlmap, Burp Suite and SonarQube; a few points, including the evaluation of the AI assistant, remain to be validated.



Keywords: client portfolio management, secure web platform, AI assistant, data protection.